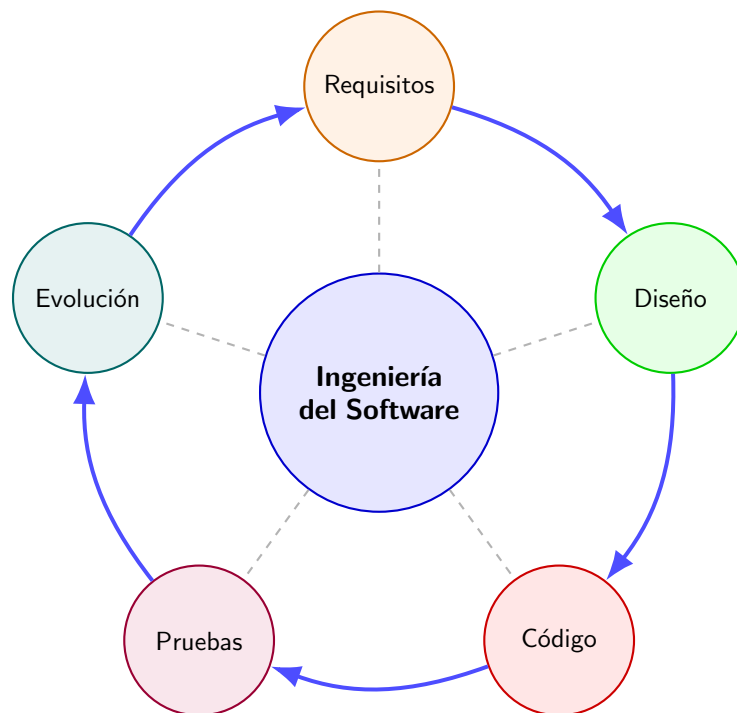


Ingeniería del Software

Apuntes Completos del Curso - Università degli Studi di Napoli Federico II



Doble Grado en Ingeniería Informática y Matemáticas (DGIIM)

Universidad de Granada / Università Federico II

Docente: Prof.ssa Anna Rita Fasolino
Estudiante: Erasmus DGIIM

21 de junio de 2026

Índice general

Capítulo 1

Introducción a la Ingeniería del Software

1.1. Naturaleza y Relevancia del Software

En el paradigma actual, nuestras vidas están profundamente digitalizadas. El software no es un mero accesorio del hardware, sino el motor ubicuo de la sociedad de la información que orquesta desde sistemas de uso personal hasta infraestructuras críticas (sistemas médicos, aviación, finanzas). Esta penetración masiva trae consigo un aumento exponencial en la complejidad técnica y en la criticidad de los sistemas.

Idea Clave: La Paradoja de los Costes

Históricamente, el hardware era el componente limitante y costoso. Hoy, la inversión económica recae abrumadoramente sobre el software. En los proyectos modernos, el desarrollo y mantenimiento del software consumen la mayor parte del coste total. Además, el coste de la evolución o mantenimiento suele superar holgadamente al coste de desarrollo inicial (pudiendo representar hasta el 60 % – 80 % del coste del ciclo de vida).

1.1.1. Contexto Histórico: La Crisis del Software

Es fundamental comprender que históricamente muchos proyectos de software fallaban no por falta de habilidad de programación de los individuos, sino por la ausencia de un marco de trabajo metodológico. Esta situación desembocó en la denominada **Crisis del Software**.

El término “*Ingeniería del Software*” fue propuesto por primera vez en las **conferencias de la OTAN de 1968 y 1969**. La idea fundamental que surgió de estas reuniones fue un cambio de paradigma radical: el desarrollo de software no es un arte ni debe ser considerado “magia”, sino que **el software debe construirse con el mismo rigor sistemático con el que se construyen los puentes**.

Durante esta crisis (que impulsó la creación de la disciplina), se identificaron empíricamente los siguientes síntomas en la industria:

- Los proyectos superaban enormemente los presupuestos asignados (llegando a costar hasta 10 veces más de lo previsto).
- Las entregas sufrían retrasos sistémicos.
- El software resultante era altamente ineficiente y de baja calidad.
- Frecuentemente no se satisfacían los requisitos originales solicitados por el cliente.

- Los proyectos se volvían inmanejables y el código (especialmente en sistemas *legacy*) era extremadamente difícil de mantener.
- En los peores casos, el software jamás llegaba a entregarse.

De esta crisis sistémica nace la necesidad ineludible de aplicar principios formales de ingeniería al desarrollo de software.

1.1.2. Consecuencias de la Falta de Ingeniería: Fallos Históricos

Para ilustrar la criticidad de aplicar estos principios rigurosos, es necesario estudiar los fallos catastróficos empíricos que ocurren cuando el software falla en entornos críticos:

- **Ámbito Médico (Therac-25, 1985–1987):** Una máquina de radioterapia que, debido a condiciones de carrera (errores de concurrencia en el software) y una mala interfaz de usuario, administró sobredosis letales de radiación a varios pacientes.
- **Ámbito Aeroespacial (Ariane 5 - Vuelo 501, 1996):** La destrucción del cohete segundos después del despegue, causando pérdidas multimillonarias. Las causas técnicas y metodológicas exactas fueron:
 1. *Testing inadecuado:* La especificación no contenía datos de la trayectoria. En las pruebas de validación, los módulos que medían la altitud fueron simulados en lugar de usar datos reales.
 2. *Reutilización incorrecta:* Se reutilizó código heredado del Ariane 4 (que funcionaba perfectamente en su entorno original). Sin embargo, el Ariane 5 tenía una aceleración horizontal mucho más rápida, lo que provocó un desbordamiento de variables (*integer overflow*) al intentar convertir un número de 64 bits de coma flotante a un entero de 16 bits.
 3. *Filosofía de diseño errónea:* Se asumió ingenuamente que, si algo fallaba, sería debido a un error de hardware aleatorio, sin prever que pudiera existir un error intrínseco de diseño en el software.

1.2. Definición de Ingeniería del Software (IS)

Antes de definir la disciplina, es imperativo establecer formalmente qué constituye el “software”:

Definición 1.1 (Software - IEEE). El **Software** no es solo código ejecutable, sino el “conjunto de programas, procedimientos, reglas y cualquier otra documentación relativa al funcionamiento” de un sistema informático.

En cuanto a la disciplina en sí, existen múltiples aproximaciones formales. Las dos más relevantes para el estudio riguroso son:

Definición 1.2 (Ingeniería del Software - Sommerville). Según Sommerville, la **Ingeniería del Software** es una disciplina de la ingeniería que se ocupa de *todos los aspectos* de la producción de software, desde las etapas iniciales de especificación del sistema hasta el mantenimiento del mismo después de que entra en uso.

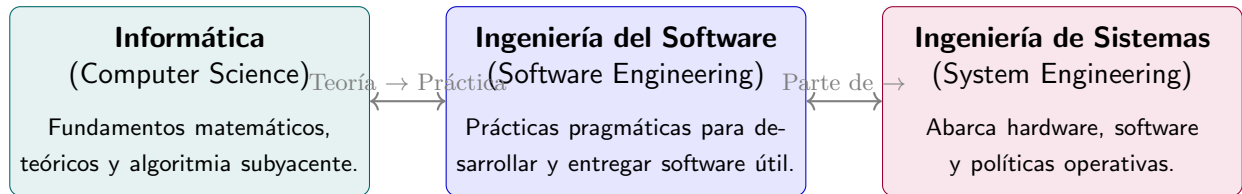
Definición 1.3 (Ingeniería del Software - IEEE). Es la aplicación de un enfoque **sistemático, disciplinado y cuantificable** al desarrollo, operación y mantenimiento del software.

Desglosemos estas definiciones para entender su rigor:

- **Disciplina Ingenieril:** Implica aplicar teorías, métodos y herramientas de forma selectiva y pragmática, tomando decisiones sujetas a restricciones económicas, temporales y organizacionales.

- **Todos los aspectos / Sistemático:** Significa que la IS trasciende la codificación. Incluye la gestión de equipos, licitación de requisitos, diseño arquitectónico, pruebas automatizadas y control de calidad (aplicando métricas cuantificables).

Es imperativo establecer las fronteras formales entre la Ingeniería del Software y otras disciplinas afines:



1.3. La Transición: De la Programación a la Ingeniería

Es un error conceptual grave confundir la mera escritura de código (*coding*) con la ingeniería. Para formalizar la disciplina, debemos distinguir claramente entre dos paradigmas de desarrollo:

- **Software Amateur:** Es aquel que es utilizado únicamente por su propio autor. No requiere de guías extensas, ni documentos formales de diseño arquitectónico, y no está sujeto a estándares estrictos de calidad o auditorías.
- **Software Profesional:** Es el código que será utilizado, mantenido y modificado por otras personas (otros ingenieros y clientes). Requiere características matemáticas y estructurales estrictas de calidad, y exige proporcionar información complementaria (documentos de diseño, manuales, *tests*) más allá del propio código fuente.

Esta dicotomía se puede resumir formalmente en la siguiente tabla comparativa:

Característica	Programación (Amateur)	Ingeniería del Software (Profesional)
Escala	Sistemas pequeños y enfocados	Sistemas complejos y a gran escala
Equipo	1 solo programador	Equipos multidisciplinares
Requisitos	Dictados por el propio desarrollador	Dictados rigurosamente por el cliente
Entregas	Una única entrega final (<i>single release</i>)	Múltiples entregas (<i>deliverables</i>)
Mantenimiento	Vida útil corta, pocas alteraciones	Vida útil larga, modificaciones frecuentes
Coste	Desarrollo prácticamente gratuito	Desarrollo altamente costoso (inversión)

1.4. Tipología de Aplicaciones Software

No existe un conjunto universal de técnicas de ingeniería de software que sea aplicable a todos los sistemas. Las herramientas y métodos que elegimos dependen matemáticamente del tipo de aplicación que estemos construyendo. Los principales tipos incluyen:

1. **Aplicaciones Independientes (*Stand-alone*):** Se ejecutan en un ordenador local sin requerir conexión a red continua (ej. aplicaciones ofimáticas, programas de edición fotográfica CAD).
2. **Aplicaciones Interactivas Basadas en Transacciones:** Se ejecutan en un ordenador remoto (servidor o nube) y los usuarios acceden desde sus terminales (ej. aplicaciones web de e-commerce, sistemas bancarios).
3. **Sistemas de Control Embebido:** Sistemas de software que controlan y gestionan dispositivos de hardware. Tienen restricciones estrictas de tiempo real (ej. software de frenos ABS, marcapasos).
4. **Sistemas de Procesamiento por Lotes (*Batch processing*):** Diseñados para procesar grandes cantidades de datos sin intervención humana interactiva (ej. sistemas de facturación de nóminas, procesamiento de datos en el CERN).
5. **Sistemas de Sistemas:** Sistemas enormemente complejos compuestos por la interacción de otros sistemas de software independientes (ej. plataformas militares de defensa).

1.5. El Producto Software y Atributos de Calidad

Un **Producto Software** no se limita al código fuente ejecutable. Comprende los programas, los archivos de configuración, la documentación del sistema y los manuales de usuario.

Según su naturaleza comercial, los productos software se clasifican en dos categorías disjuntas:

- **Productos Genéricos:** Sistemas producidos por una organización para ser vendidos en el mercado abierto a cualquier cliente (ej. editores de texto, bases de datos comerciales, sistemas operativos). Las especificaciones son dictadas por el desarrollador.
- **Productos Personalizados (Bespoke):** Sistemas diseñados bajo demanda para un cliente específico según sus necesidades particulares (ej. sistemas de control aéreo, software hospitalario). Las especificaciones son dictadas por el cliente.

1.5.1. La Naturaleza Oculta de los Defectos de Calidad

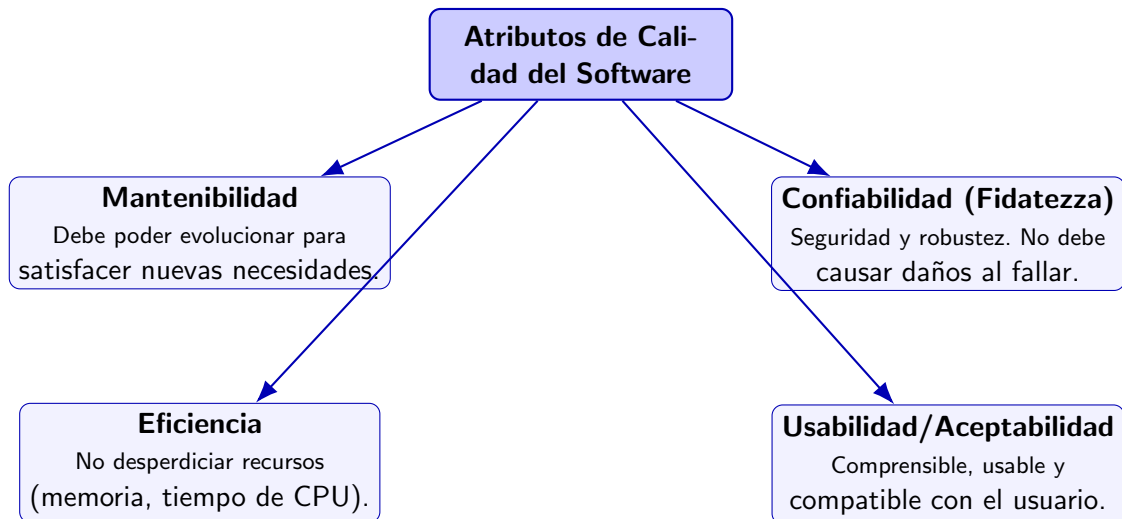
Un corolario importante en la ingeniería de producto es la asimetría temporal en el ciclo de vida de los errores. Los problemas de calidad del software son inherentemente difíciles de detectar en el momento exacto en que se cometen.

Idea Clave: El Ciclo de Vida del Error

La inmensa mayoría de los defectos estructurales se introducen en las fases iniciales del proyecto (durante la especificación de requisitos y el diseño arquitectónico). Sin embargo, no se descubren hasta mucho más tarde, típicamente cuando el software ya está en la fase operativa. En este punto, el coste matemático y logístico de reparar un defecto se multiplica exponencialmente.

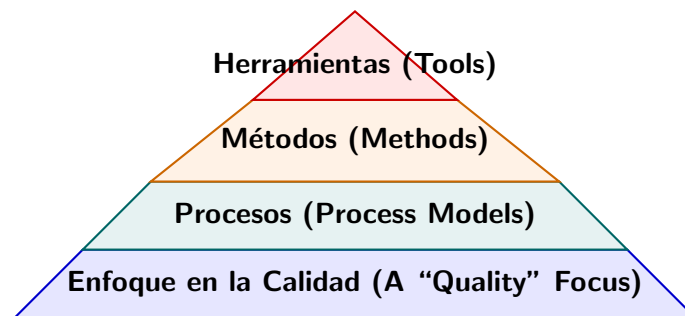
1.5.2. Atributos de Calidad (McCall / Sommerville)

El software bien diseñado debe ir más allá de simplemente calcular la salida correcta; debe exhibir atributos de calidad (requisitos no funcionales) críticos. Los cuatro atributos principales son:



1.6. Procesos, Métodos y Herramientas: El Modelo Piramidal

Para producir software de manera predecible y evitar los fallos sistémicos, la Ingeniería del Software estructura sus fundamentos jerárquicamente a través de un **Modelo Piramidal**.



Desglosemos esta jerarquía desde la base hasta la cúspide:

1. **La Base (El Enfoque en la Calidad):** Toda la ingeniería se cimenta axiomáticamente sobre el compromiso continuo con la calidad. Si el objetivo principal no es construir software robusto y seguro, el resto de la pirámide colapsa.
2. **Procesos:** Sobre la calidad se asientan los modelos de proceso (que definen las fases y el cronograma general de actividades).
3. **Métodos:** Dentro de los procesos, aplicamos los métodos (las reglas, técnicas y el "cómo" detallado para construir los modelos y el código).
4. **Herramientas:** En la cima están las herramientas CASE automatizadas que dan soporte tecnológico y aligeran la carga de ejecución de los métodos y procesos.

1.6.1. El Proceso Software

Un proceso de software es el conjunto estructurado de actividades requeridas para desarrollar un sistema. Independientemente del paradigma o modelo (Cascada, Ágil, etc.), todo proceso involucra cuatro actividades fundamentales:

1. **Especificación (Requisitos):** Definir estrictamente qué debe hacer el sistema y cuáles son sus restricciones operativas.
2. **Desarrollo (Diseño e Implementación):** Transformar la especificación en un producto ejecutable definiendo su arquitectura.
3. **Validación (Pruebas/Testing):** Comprobar mediante baterías de pruebas y revisiones que el software cumple con lo exigido por el cliente.
4. **Evolución (Mantenimiento):** Modificar el sistema en respuesta a cambios en las dinámicas del mercado o del entorno tecnológico.

1.6.2. Métodos de la Ingeniería del Software

Un método es una aproximación estructurada al desarrollo, orientada a facilitar la producción sistémica de calidad. Un método estándar (*methodology*) típicamente incluye:

- **Modelos de Sistema:** Notaciones gráficas o formales (como los diagramas UML) para representar las abstracciones del sistema.
- **Reglas:** Restricciones formales que los modelos y el código deben respetar.
- **Recomendaciones:** Consejos heurísticos o patrones de diseño probados.
- **Guía de Procesos:** Definición de las actividades a realizar, los artefactos a producir y los roles del equipo.

1.6.3. Herramientas CASE (*Computer-Aided Software Engineering*)

El soporte tecnológico para las actividades del proceso se implementa a través de herramientas CASE, concebidas para reducir el trabajo repetitivo y minimizar errores de integración.

Recuerdo: Clasificación CASE

- **Upper-CASE:** Orientadas a las fases tempranas del ciclo de vida. Apoyan la licitación de requisitos, el análisis funcional y el modelado/diseño arquitectónico UML (ej. *Visual Paradigm*).
- **Lower-CASE:** Orientadas a las fases tardías de construcción. Dan soporte a la programación, el debugging, el control de versiones continuas y el testing automatizado (ej. *GitHub*, *JUnit 5*, *SonarQube*).

1.7. Ética y Responsabilidad Profesional

La ingeniería del software, dada su capacidad de impacto masivo, exige de sus practicantes un comportamiento técnico irreprochable. Los ingenieros no solo deben velar por el rendimiento del algoritmo, sino por su implicación ética. Las normativas internacionales delinean áreas de responsabilidad clave:

- **Confidencialidad:** Un ingeniero debe respetar sistemáticamente la información confidencial de sus empleadores o clientes, exista o no un contrato formal de confidencialidad (NDA).

- **Competencia Profesional:** Es una violación ética aceptar conscientemente trabajos críticos que excedan nuestro nivel de competencia técnica o matemática sin advertirlo a las partes implicadas.
- **Derechos de Propiedad Intelectual (IP):** Se debe asegurar la estricta observancia de las leyes de patentes, *copyright* y proteger el código fuente como un activo valioso.
- **Uso Adecuado de Ordenadores:** Un profesional nunca debe emplear sus privilegios de acceso para vulnerar sistemas ajenos, introducir puertas traseras o diseminar software malicioso, incluso como “prueba”.

Capítulo 2

Ciclo de Vida del Software

2.1. Conceptos Fundamentales

Para abordar la construcción de sistemas complejos, la ingeniería requiere de estructuras formales que permitan organizar las actividades en el eje temporal.

Definición 2.1 (Ciclo de Vida - ISO/IEC/IEEE 12207:2017 y Scacchi). El **Ciclo de Vida del Software** (CVS o *Software Life Cycle*, SLC) se define normativamente como “la evolución de un sistema, producto, servicio, proyecto u otra entidad creada por el hombre, desde su concepción hasta su retirada (*retirement*)”.

Adicionalmente, W. Scacchi lo describe como una caracterización descriptiva o prescriptiva de cómo un sistema software viene o dovrebbe essere sviluppato.

Modelar este proceso es crítico para los ingenieros. Todo modelo de ciclo de vida formal persigue **tres objetivos fundamentales**:

1. **Determinación del orden:** Establecer la secuencia cronológica y lógica de ejecución de las actividades.
2. **Criterios de transición:** Definir las condiciones exactas (hitos o *milestones*) que permiten pasar de una fase a la siguiente.
3. **Definición de responsabilidades:** Asignar qué roles producen qué entregables (*deliverables*) a lo largo del proceso.

Además, el uso de un modelo proporciona un marco general para:

- **Comprensión:** Establecer un vocabulario común entre el equipo y el cliente.
- **Control:** Estandarizar las prácticas y definir hitos observables (*milestones*).
- **Estimación:** Facilitar el cálculo del coste (esfuerzo) y los plazos temporales.

Es imperativo distinguir matemáticamente entre **Fases** y **Actividades**:

- **Fase:** Es un bloque estrictamente temporal del proyecto (ej. “Mes 1 a Mes 3”).
- **Actividad / Disciplina:** Es un esfuerzo lógico o técnico (ej. “Escribir código”, “Diseñar base de datos”). Una actividad puede distribuirse transversalmente a lo largo de múltiples fases.

2.1.1. El Anti-Modelo: Escribir y Arreglar (*Code and Fix*)

Antes de estudiar los modelos formales, es crucial identificar el enfoque diametralmente opuesto a la ingeniería, referenciado habitualmente como **Code and Fix** (Escribir código y arreglar).

En este enfoque, el desarrollador salta directamente a la fase de codificación sin ningún análisis previo, diseño arquitectónico ni especificación de requisitos. A medida que surgen errores o el cliente solicita cambios, el código se modifica parcheándolo empíricamente sobre la marcha.

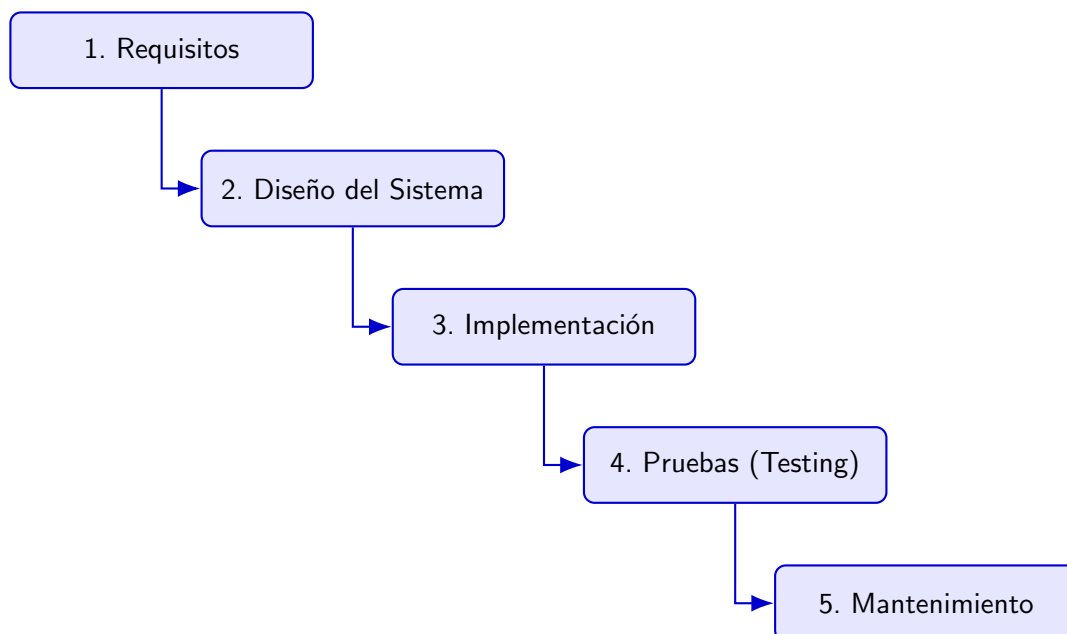
Observación 2.2 (Ausencia de Modelo). En la disciplina de la Ingeniería del Software, la profesora destaca explícitamente que el enfoque *Code and Fix* **no se considera un modelo de ciclo de vida**. Carece de planificación, no define fases ni criterios de transición, y su aplicación produce sistemas inmanejables, opacos para los gerentes e imposibles de mantener. Solo es admisible en el estricto contexto del *Software Amateur*.

2.2. Modelos de Proceso Secuenciales

Los modelos secuenciales asumen un progreso determinista y lineal. Son la traslación directa de las metodologías de la ingeniería civil y mecánica al desarrollo de software.

2.2.1. El Modelo en Cascada (Waterfall)

Introducido por W. Royce en 1970, considera que el desarrollo fluye de forma lineal y estrictamente descendente. La salida documentada de una fase es la entrada inmutable de la siguiente.



Análisis Crítico:

- **Ventajas:** Estructura lógica clara. Genera una documentación exhaustiva. Ideal para sistemas críticos (ej. *software aeroespacial*) donde los requisitos están matemáticamente demostrados desde el inicio.
- **Desventajas (Inflexibilidad):** Extrema rigidez. Asume el irreal escenario de que los requisitos no cambiarán. El cliente no visualiza software funcional hasta la etapa 4, incurriendo en un alto riesgo de decepción.

Variante en la Práctica (Cascada con Feedback): En la realidad de la industria, los modelos en cascada puros no existen. En la práctica se utiliza una variante que incluye **flechas de retroalimentación (feedback)** que permiten volver a la fase inmediatamente anterior cuando se detecta un error. Sin embargo, la profesora subraya que aunque esto lo hace viable, cada iteración de retroceso altera la planificación y encarece exponencialmente el proyecto.

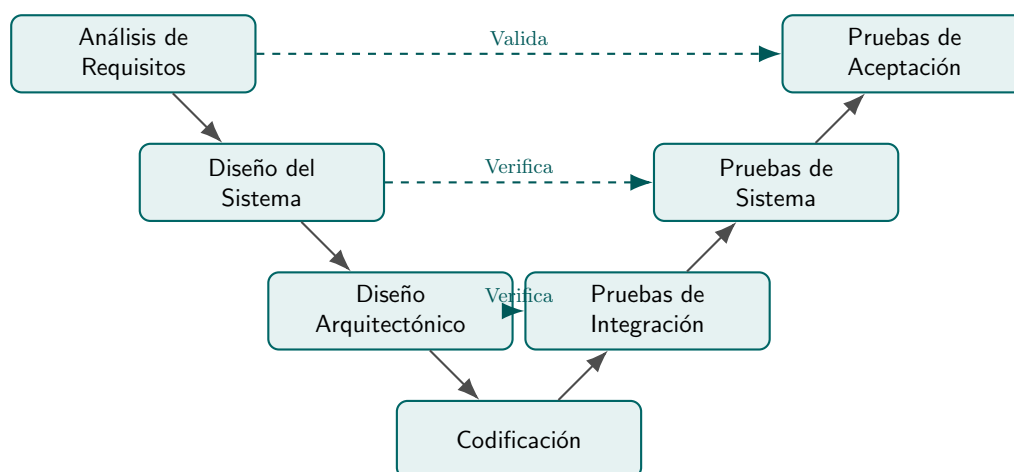
2.2.2. El Modelo en V

Es una extensión estructurada de la Cascada que solventa el problema de dejar las pruebas para el final. Postula que por cada fase de *desarrollo* (rama descendente) debe planificarse inmediatamente su correspondiente fase de *prueba* (rama ascendente).

Idea Clave: Validación vs. Verificación (V&V)

El modelo en V se basa en dos conceptos ortogonales:

- **Verificación:** “¿Estamos construyendo el producto correctamente?” (El código cumple la especificación técnica).
- **Validación:** “¿Estamos construyendo el producto correcto?” (El sistema resuelve la necesidad real del cliente).



2.3. Modelos Evolutivos e Iterativos

En la práctica, los requisitos de un sistema son funciones dependientes del tiempo, $R = f(t)$, debido a la volatilidad del mercado. Los modelos evolutivos abrazan este dinamismo.

Recuerdo: Iterativo vs. Incremental

Aunque suelen usarse juntos (Desarrollo Iterativo-Incremental), semánticamente son distintos:

- **Iterativo:** Refinar progresivamente una característica. Es análogo a esculpir una estatua: primero se hace un boceto general, luego se tallan los detalles.
- **Incremental:** Añadir funcionalidad modularmente completa. Es análogo a construir una casa habitación por habitación; cada habitación añadida es 100 % funcional.

2.3.1. Prototipación (Prototyping)

Consiste en construir un modelo inicial, rápido y ejecutable del software. Permite al cliente experimentar con el sistema y refinar sus requisitos reales antes de la costosa fase de implementación. Puede ser **Exploratoria** (evoluciona hasta convertirse en el sistema final) o de **Usa y Tira** (*Throw-away*: se descarta tras entender el problema para programarlo desde cero de forma robusta).

Idea Clave: El Defecto Crítico: Falta de Visibilidad

La profesora destaca explícitamente que el mayor problema estructural de los modelos evolutivos puros es la **Falta de Visibilidad** (*mancanza di visibilità*). Al no existir la obligación de producir documentos formales exhaustivos al final de cada iteración corta, los gerentes (*managers*) del proyecto no pueden medir el progreso real del sistema de forma cuantificable tradicional.

2.4. Modelos Dirigidos por Riesgos: La Espiral de Boehm

Propuesto por Barry Boehm, este modelo representa el proceso como una espiral. A diferencia de otros enfoques dirigidos por documentos o por código, la Espiral está explícitamente **dirigida por la evaluación del riesgo** (*Risk-driven*).

Cada iteración (un bucle de 360° en la espiral) aborda un conjunto de riesgos y se divide matemáticamente en cuatro cuadrantes rigurosos:

1. **Definición de Objetivos:** Se identifican las metas específicas de la fase, las alternativas tecnológicas disponibles y las restricciones aplicables.
2. **Evaluación y reducción de riesgos:** Es el paso central y definitorio del modelo. Se analizan meticulosamente los riesgos identificados (por ejemplo, requisitos inciertos, cuellos de botella de rendimiento) y se toman medidas para mitigarlos (frecuentemente construyendo un prototipo técnico específico).
3. **Desarrollo y validación:** Una vez evaluado el riesgo, se elige el modelo de desarrollo más apropiado *solo para ese bucle* (se puede elegir cascada, evolutivo, etc.) y se construye/verifica el software correspondiente.
4. **Planificación:** Revisión con el cliente de los resultados del bucle y planificación estratégica en detalle de la siguiente fase de la espiral.

2.5. El Proceso RUP (Rational Unified Process)

RUP es un marco de proceso moderno que se sitúa a medio camino entre la pesadez de la Cascada y la ligereza de las Metodologías Ágiles. Se caracteriza por estar **dirigido por Casos de Uso** y **centrado en la Arquitectura**.

RUP es bidimensional. Se representa matemáticamente mediante una matriz donde el eje *X* representa el tiempo (*Fases*) y el eje *Y* representa el esfuerzo lógico (*Disciplinas*).

Las 4 Fases Temporales de RUP:

- **Inception (Inicio):** Viabilidad económica del proyecto (*Business Case*) y delimitación del alcance.
- **Elaboration (Elaboración):** Captura de los requisitos principales, eliminación de los mayores riesgos arquitectónicos y establecimiento de un esqueleto base del sistema.
- **Construction (Construcción):** Desarrollo iterativo del grueso del código y ensamblaje de los componentes.

- **Transition (Transición):** Despliegue del software a los usuarios finales (beta testing, corrección de errores menores, capacitación).

2.6. Criterios de Selección y Análisis de Riesgos

La selección del modelo de ciclo de vida no es arbitraria, sino que depende del perfil de riesgo del proyecto. Según las diapositivas de la profesora, la asignación se resume en:

- Modelo en Cascada:

- *Alto riesgo:* Para sistemas nuevos con requisitos no claros o fluctuantes.
- *Bajo riesgo (Basso rischio):* Para sistemas con especificaciones muy estables y tecnologías sólidamente consolidadas (*assestate*).

- Modelos Evolutivos:

- *Alto riesgo:* Organizativo, debido a la grave falta de visibilidad del estado real del proyecto.
- *Bajo riesgo (Basso rischio):* Para aplicaciones totalmente nuevas, entornos con variables ambientales y donde se requiere una alta evolutividad.

- Modelo en Espiral:

- Ideal para proyectos donde cada iteración viene caracterizada por riesgos específicos (personal, presupuesto, funcionalidades complejas, interfaces inadecuadas) que deben mitigarse de forma explícita antes de programar.

Ejemplo 2.3 (Elección de un Modelo de Ciclo de Vida). **Contexto:** Una empresa (*SoftwareDGIIM S.A.*) ha sido contratada por la Agencia Espacial Europea (ESA) para desarrollar un módulo de control de propulsión de un satélite. Paralelamente, una Startup (*AppNapoli*) les pide una aplicación móvil de entrega de comida a domicilio cuya competencia es feroz y los requisitos cambian cada semana.

Problema: Justificar formalmente qué ciclo de vida asignar a cada proyecto.

Resolución Analítica:

1. Proyecto ESA (Satélite):

- *Caracterización:* Riesgo extremo para la vida humana y la inversión material. Requisitos inmutables y matemáticamente demostrables.
- *Selección:* **Modelo en V** (o Cascada rigurosa).
- *Justificación:* Permite asegurar que cada línea de código (codificación) tenga asociada una prueba formal y validada en etapas tempranas. Se prefiere la robustez y verificación exhaustiva antes que la velocidad de entrega.

2. Proyecto Startup (App Móvil):

- *Caracterización:* Tiempo de llegada al mercado (*Time-to-Market*) crítico. Requisitos altamente volátiles, dependientes de métricas de usuario.
- *Selección:* **Modelo Evolutivo-Incremental (Ágil).**
- *Justificación:* El cliente necesita ver prototipos funcionales rápidos. Las iteraciones cortas permitirán pivotar las funcionalidades en respuesta al mercado sin desechar grandes cantidades de planificación.

Capítulo 3

Metodologías Ágiles: Scrum, XP y DevOps

3.1. La Crisis Estructural y el Paradigma Ágil

Los modelos prescriptivos clásicos (como la Cascada) asumen que es matemáticamente posible elicitar y congelar todos los requisitos de un sistema de forma determinista al inicio del proyecto. Sin embargo, la evidencia empírica demuestra que, en entornos de innovación, los requisitos son variables dependientes del tiempo.

Según el *CHAOS Report* del *Standish Group* (referenciado en las clases), una proporción alarmante de proyectos tradicionales fracasan:

- Solo alrededor del 34 % de los proyectos tienen éxito total.
- Apenas un 52 % de las funcionalidades solicitadas originalmente llegan a producirse y utilizarse realmente por el usuario final.

Para mitigar esta ineficiencia y reducir la tasa de fracaso en proyectos con requisitos volátiles, surge el paradigma **Ágil**. La insatisfacción por la extrema burocracia de los modelos tradicionales llevó a un grupo de ingenieros a formalizar este movimiento en **febrero de 2001 en Utah**.

Las metodologías más difundidas bajo este paradigma incluyen: eXtreme Programming (XP), Scrum, Feature Driven Development (FDD), Dynamic Systems Development Method (DSDM), Crystal y, como evolución natural, DevOps.

Idea Clave: El Manifiesto Ágil

Los métodos ágiles no rechazan la ingeniería, sino que reordenan sus prioridades. Sus cuatro valores axiomáticos priorizan:

1. **Individuos e interacciones** sobre procesos y herramientas.
2. **Software funcionando** sobre documentación exhaustiva.
3. **Colaboración con el cliente** sobre negociación contractual.
4. **Respuesta ante el cambio** sobre el seguimiento estricto de un plan.

3.2. eXtreme Programming (XP)

Extreme Programming (XP), introducido a finales de los 90, es la metodología ágil más orientada puramente al código y a las prácticas técnicas de la ingeniería de software. XP lleva las metodologías

iterativas a casos “extremos”, basándose en el mejoramiento constante y continuo del código. Toda la arquitectura se centra en la iteración actual y no en necesidades futuras inciertas.

El núcleo de XP se apoya en **12 prácticas técnicas formales**, que operan en sinergia:

1. **Planning Game:** Determina el objetivo y los tiempos de la próxima entrega basándose en prioridades de negocio.
2. **Small Releases:** Liberar rápidamente pequeñas actualizaciones o incrementos de funcionalidad.
3. **Metaphor (Metáfora):** Guiar el desarrollo a través de una “historia” o visión compartida que describe cómo funciona todo el sistema.
4. **Simple Design:** El sistema debe diseñarse de la forma más sencilla posible que funcione en el momento presente.
5. **Refactoring Continuo:** Mejorar constantemente el diseño interno del código fuente sin alterar su comportamiento externo.
6. **Testing (TDD):** Efectuar pruebas constantemente. Se escriben los tests automáticos *antes* de escribir la funcionalidad matemática que los supera (Test-First Development).
7. **Pair Programming:** Todo código de producción es escrito por dos ingenieros en una misma máquina, garantizando revisión continua y transferencia de conocimiento.
8. **Collective Ownership:** Cualquier ingeniero puede cambiar cualquier parte del código en cualquier momento; el código es del equipo, no de individuos.
9. **Continuous Integration (CI):** El software se compila, integra y testea automáticamente múltiples veces al día, previniendo el infierno de integración.
10. **40-Hour Week:** Trabajar a un ritmo sostenible (no más de 40 horas semanales) para evitar el agotamiento y mantener la calidad del software.
11. **On-site Customer:** Un representante real del cliente debe estar disponible a tiempo completo para el equipo de desarrollo.
12. **Coding Standards:** Aplicar reglas y estándares estrictos para que el código parezca escrito por una sola persona.

3.3. El Framework Scrum

Scrum es un marco de trabajo empírico y ágil diseñado para gobernar la imprevisibilidad. Matemáticamente, Scrum no prescribe cómo resolver un problema, sino que establece un meta-modelo de control de procesos basado en la teoría del empirismo: la **transparencia**, la **inspección** y la **adaptación** iterativa.

Estructuralmente, el ciclo de vida de un proyecto Scrum se divide en **tres fases macro**:

1. **Pre-game phase:** Incluye la sub-fase de planificación y la sub-fase de arquitectura inicial.
2. **Development (Game) phase:** El sistema se desarrolla a través de una serie iterativa de Sprints.
3. **Post-game phase:** Contiene la clausura definitiva y liberación (*release*) del sistema.

La unidad básica de tiempo en la fase de desarrollo es el **Sprint**. Un Sprint es un contenedor temporal estrictamente acotado (*timeboxed*), habitualmente de 2 a 4 semanas, durante el cual se crea un incremento de producto terminado y potencialmente desplegable.

3.3.1. Tipología Vectorial de Scrum

El framework Scrum se define íntegramente por un conjunto cerrado de 3 Roles, 3 Artefactos y 5 Eventos formales.

1. Los Roles (El Equipo Scrum)

- **Product Owner (PO):** Es la voz del cliente. Maximiza el valor del producto gestionando rigurosamente el *Product Backlog*. Es una única persona, no un comité.
- **Scrum Master (SM):** Es el ingeniero responsable de que el equipo comprenda y aplique la topología y reglas de Scrum. Elimina impedimentos técnicos u organizativos.
- **Developers (El Equipo de Desarrollo):** Grupo de profesionales (típicamente de **5 a 9 personas**) con organización autónoma y habilidades cruzadas (*cross-functional*). En Scrum **no existen títulos ni sub-roles** (arquitecto, tester) dentro de este equipo; todos son desarrolladores.

2. Los Artefactos (Estado del Sistema)

- **Product Backlog:** Una lista priorizada y dinámica de todo lo que podría ser necesario en el producto final. Los elementos suelen redactarse como *User Stories*.
- **Sprint Backlog:** Subconjunto del Product Backlog extraído para el Sprint actual, junto con el plan para transformarlo en software funcional.
- **Incremento:** La suma de todos los elementos del Product Backlog completados y validados durante un Sprint.

3. Los 5 Eventos Formales (Sincronización)

- **El Sprint:** Es el evento contenedor. Todos los demás eventos ocurren dentro de él.
- **Sprint Planning:** Reunión al inicio del Sprint para definir qué se construirá (objetivo) y cómo se hará matemáticamente.
- **Daily Scrum:** Sincronización diaria estricta de 15 minutos del equipo de desarrollo. Se deben responder 3 preguntas fijas: 1) *¿Qué hice ayer?* 2) *¿Qué haré hoy?* 3) *¿Hay algún impedimento bloqueando mi trabajo?*
- **Sprint Review:** Inspección del Incremento construido al final del Sprint, adaptando el Backlog según el *feedback* del cliente.
- **Sprint Retrospective:** Análisis interno del equipo sobre sus métricas de eficiencia, procesos y herramientas para aplicar mejoras en el siguiente bucle.

3.3.2. Historias de Usuario (User Stories)

En Agile, los requisitos no se redactan como pesados documentos SRS, sino como **User Stories**. Son descripciones cortas y funcionales escritas desde la perspectiva del usuario final.

Idea Clave: Estructura Canónica de una User Story

Toda User Story profesional debe adherirse a la siguiente plantilla sintáctica:

“Como [rol de usuario / actor], quiero [funcionalidad concreta / acción], para poder [objetivo empresarial o de usuario / beneficio].”

3.4. El Paradigma DevOps

Mientras que el movimiento Ágil resolvió el cuello de botella entre negocio y desarrollo (el código se escribía rápido), generó un nuevo problema: los ingenieros de sistemas/operaciones (*Ops*) no podían desplegar en producción el software a la misma velocidad que los desarrolladores (*Devs*) lo programaban.

La profesora introduce **DevOps** como la maduración natural del ecosistema Ágil.

Definición 3.1 (DevOps). DevOps es un conjunto de prácticas, cultura y herramientas diseñadas para colmar la brecha (*gap*) entre el desarrollo del software (*Development*) y la fase operativa o de sistemas (*Operations*).

Su meta matemática es reducir el tiempo entre que un desarrollador escribe una línea de código (*commit*) y esa línea se ejecuta de forma segura y productiva para el cliente final.

3.4.1. Principios y Prácticas CAMS

DevOps se rige por un marco conceptual conocido como **CAMS**:

- **Culture** (Cultura): Énfasis en la colaboración, empatía y comunicación inter-equipos (romper los “silos”).
- **Automation** (Automatización): Eliminar el factor de error humano. Automatizar construcción (*Build*), integración (*Integration/Test*), despliegue (*Deploy*) y monitoreo.
- **Measurement** (Medición): Soporte en datos objetivos y telemetría para análisis de tendencias y calidad técnica.
- **Sharing** (Compartir): Retroalimentación iterativa continua.

3.4.2. El Pipeline CI/CD

En la infraestructura DevOps, la automatización se materializa a través de herramientas de la cadena (*toolchain*), destacando el paradigma CI/CD:

- **Continuous Integration (CI)**: Herramientas (ej. *Jenkins*, *GitLab CI*) que ensamblan automáticamente el código de varios desarrolladores y ejecutan test unitarios para detectar fallos inmediatamente tras cada modificación.
- **Continuous Deployment / Delivery (CD)**: Despliegue automatizado del software validado directamente hacia los servidores de producción o pre-producción mediante el uso de contenedores virtuales (ej. *Docker*, *Kubernetes*).

Ejemplo 3.2 (Simulación Analítica de un Sprint: Ticket Eventos). **Contexto:** El equipo de ingenieros de DGIIM está desarrollando en Scrum un *software* para la venta de billetes de eventos online y gestión de puntos de fidelidad. El equipo tiene una “velocidad” estimada de 10 Story Points por Sprint.

Problema 1: Redacción de User Stories El *Product Owner* te pide que formalices el requisito: “El sistema debe dar un billete gratis si el usuario tiene 100 puntos”.

Solución: Utilizando el formato estándar: “Como [*Cliente recurrente*], quiero [*poder canjear 100 de mis puntos acumulados*], para [*obtener un billete de acceso gratuito a un evento*].”

Problema 2: Gestión de Cambio de Requisitos Durante el día 5 del Sprint (cuyo objetivo es entregar el carrito de compras), el cliente envía un correo urgente: “Necesito que los usuarios puedan filtrar los eventos solo por aquellos que tengan entradas disponibles hoy mismo”. ¿Debe el equipo detener lo que está haciendo e incluir esta funcionalidad en el Sprint actual?

Solución Analítica: NO. Según las reglas formales de Scrum explicitadas por la profesora, **el Sprint es inmutable y estable**. Una vez que el *Sprint Backlog* ha sido comprometido en el *Sprint Planning*, no se admiten cambios externos que pongan en riesgo el *Sprint Goal*. El protocolo sistémico a seguir es: El *Product Owner* recibe la solicitud, la traduce a una *User Story*, la prioriza y la inserta en el **Product Backlog** general. El equipo la evaluará y, si tiene alta prioridad de negocio, la incluirá en la iteración del *próximo* Sprint.

Capítulo 4

Análisis y Especificación de Requisitos

4.1. La Ingeniería de Requisitos

El diseño de software no comienza con la escritura de código, sino con la comprensión matemática y formal del problema a resolver. Como postuló el ingeniero Charles F. Kettering: “*Un problema bien definido es un problema ya medio resuelto*”.

Definición 4.1. La **Ingeniería de Requisitos** es el proceso sistemático de descubrir, analizar, documentar y verificar los servicios que el cliente requiere del sistema, así como las restricciones operativas bajo las cuales dicho sistema debe operar y desarrollarse.

Un requisito puede variar desde una declaración abstracta de alto nivel de un servicio, hasta una especificación detallada y matemática de una función del sistema. Esta varianza genera dos niveles de abstracción principales:

1. **Requisitos de Usuario:** Declaraciones en lenguaje natural (apoyadas por diagramas) de los servicios que el sistema proporciona y sus restricciones operativas. Están dirigidos a un público no técnico: **clientes, gerentes de la empresa (*committenti*) y usuarios finales**.
2. **Requisitos de Sistema:** Un documento estructurado que define detalladamente las funciones, servicios y restricciones. Es un contrato técnico exacto dirigido a un público técnico: **arquitectos de software, desarrolladores y testers**.

4.2. Taxonomía de los Requisitos

Para garantizar un análisis riguroso, los requisitos del sistema se clasifican ortogonalmente en tres categorías: Funcionales, No Funcionales y de Dominio.

Idea Clave: Requisitos Funcionales vs. No Funcionales

Los requisitos **funcionales** describen *qué* debe hacer el sistema (mapeo de entradas a salidas). Los requisitos **no funcionales** describen *cómo* debe hacerlo (restricciones de rendimiento, seguridad, estándares).

4.2.1. Requisitos Funcionales (FR)

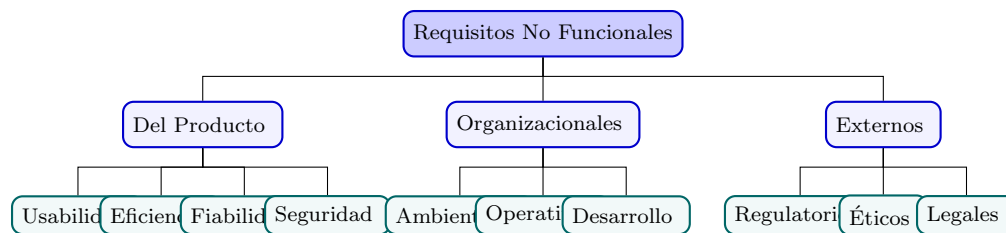
Definen los servicios que el sistema debe proporcionar, cómo debe reaccionar a entradas específicas y cómo debe comportarse en situaciones particulares. En sistemas críticos, también pueden declarar explícitamente lo que el sistema *no* debe hacer.

Ejemplo: “El sistema debe permitir a los usuarios buscar un artículo en el inventario utilizando su identificador alfanumérico o una subcadena de su descripción.”

4.2.2. Requisitos No Funcionales (NFR)

Son restricciones aplicadas a los servicios o funciones ofrecidas por el sistema. A menudo se aplican al sistema en su totalidad en lugar de a funciones individuales. Son críticos: si un FR no se cumple, el sistema está incompleto; si un NFR (como el tiempo de respuesta o la seguridad) no se cumple, el sistema puede ser totalmente inútil.

Según la clasificación de Sommerville (presentada en clase), los NFR se estructuran en el siguiente árbol jerárquico:



Métricas para hacer verificables los NFRs

Para que un NFR sea útil en ingeniería, debe ser **cuantificable**. Las métricas estándar son:

- **Velocidad (Speed):** Transacciones/segundo, tiempo de respuesta en milisegundos.
- **Tamaño (Size):** Megabytes (MB) en memoria RAM o almacenamiento ROM.
- **Facilidad de uso (Ease of use):** Tiempo de entrenamiento requerido (horas/días).
- **Fiabilidad (Reliability):** Tiempo Medio Entre Fallos (MTBF), Tasa de disponibilidad (ej. 99.99 %).
- **Robustez (Robustness):** Tiempo para reiniciar tras un fallo, porcentaje de eventos causantes de falla.
- **Portabilidad (Portability):** Porcentaje de código dependiente de la plataforma destino.

4.2.3. Requisitos de Dominio

Requisitos que provienen del dominio de aplicación del sistema y que reflejan características de dicho dominio. Pueden generar nuevos requisitos funcionales o imponer restricciones complejas sobre los existentes.

Ejemplo: En un software balístico, un requisito de dominio sería: “La desaceleración se calculará según la ecuación de resistencia aerodinámica $F_D = \frac{1}{2}\rho v^2 C_D A$ ”.

Observación 4.2 (Problemas Críticos de los Requisitos de Dominio). La literatura advierte sobre dos problemas sistémicos al tratar con estos requisitos:

1. **Comprensibilidad:** Están expresados en la jerga (*gergo*) matemática o técnica del dominio de aplicación, resultando incomprensibles para los ingenieros de software.
2. **Implicitud:** Los expertos del dominio omiten información crucial porque la consideran “conocimiento evidente o trivial”.

4.3. Especificación y Reglas de Redacción

Escribir requisitos en lenguaje natural libre introduce problemas semánticos severos (ambigüedad, confusión de conceptos, amalgama de múltiples funciones en una sola frase).

La consecuencia más grave de la ambigüedad es el **Sesgo del Desarrollador (*Developer Bias*)**: ante un requisito poco claro (ej. “buscar”), el desarrollador lo interpretará según su propio criterio técnico, produciendo un sistema que hace algo distinto a lo que el cliente imaginaba.

Para que una especificación sea rigurosa, cada requisito debe cumplir las siguientes propiedades:

- **Preciso y No Ambiguo**: Solo debe tener una interpretación lógica posible.
- **Completo**: Debe definir los servicios de forma total, incluyendo el manejo de errores.
- **Consistente**: No deben existir contradicciones lógicas entre dos requisitos ($R_1 \wedge R_2 \neq \text{Falso}$).
- **Verificable / Testable**: Debe ser posible escribir una prueba matemática o empírica para el requisito.

Idea Clave: Lineamientos de Redacción Formal

Las normativas establecen el uso diferencial de los verbos modales ingleses:

- **SHALL (Debe)**: Expresa una obligación absoluta o un requisito mandatorio.
- **SHOULD (Debería)**: Expresa un requisito deseable pero no estrictamente obligatorio.

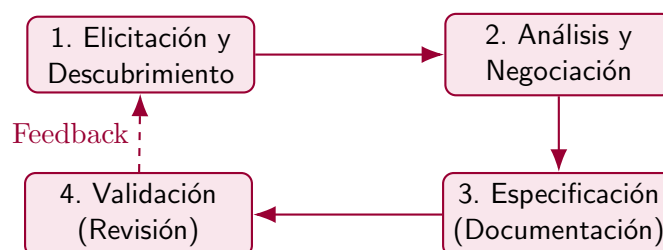
4.3.1. Alternativas al Lenguaje Natural Libre

Para mitigar la ambigüedad, la ingeniería de software emplea notaciones más restrictivas:

1. **Lenguaje Natural Estructurado**: Uso de plantillas predefinidas o formularios estándar. Minimiza el riesgo obligando al ingeniero a completar campos estrictos: *Descripción, Entradas, Origen de las entradas, Salidas, Destino de las salidas, Precondiciones y Postcondiciones*.
2. **Notaciones Gráficas**: Uso de modelos visuales semánticos, como los Diagramas UML o diagramas de secuencia.
3. **Especificaciones Matemáticas**: Notaciones formales basadas en lógica de primer orden o máquinas de estado finito (útiles en sistemas de misión crítica, aunque raras por su alto coste de comprensión).

4.4. El Proceso de Ingeniería de Requisitos

La recolección y formalización de requisitos es un proceso iterativo y cíclico compuesto por cuatro macro-actividades:



4.4.1. Técnicas de Descubrimiento (Elicitación)

Para extraer la información de los *stakeholders* (quienes a menudo no saben qué quieren exactamente o expresan los requisitos en sus propios términos), los ingenieros utilizan tres técnicas formales:

1. **Entrevistas (Interviews):** Pueden ser cerradas (cuestionarios predefinidos) o abiertas (exploratorias). Son útiles para entender el contexto general.
2. **Escenarios (Scenarios):** Descripciones narrativas paso a paso de interacciones específicas con el sistema. Son la base para los futuros Casos de Uso.
3. **Etnografía (Observación):** El ingeniero se sumerge en el entorno de trabajo del cliente observando las tareas operativas reales. Es excelente para descubrir requisitos implícitos y procesos cooperativos, pero **no es útil** para inventar nuevas funcionalidades.

4.4.2. Validación de Requisitos

Validar es demostrar que los requisitos definen el sistema que el cliente realmente quiere. Un error de requisitos detectado tarde puede costar hasta 100 veces más que si se detecta en esta fase. Las técnicas de validación incluyen:

- **Revisiones de Requisitos (Reviews):** Análisis manual y sistemático del documento por parte de un equipo de revisores multidisciplinar (cliente, analistas, desarrolladores).
- **Prototipado (Prototyping):** Construcción de un modelo ejecutable del sistema para que los usuarios finales lo experimenten empíricamente.
- **Generación de Casos de Prueba:** Escribir los tests conceptuales basándose en el documento. Si un test es imposible de diseñar analíticamente, el requisito es deficiente o ambiguo.

4.5. Gestión de Requisitos y Trazabilidad

En la realidad empresarial, los requisitos son volátiles. Suelen cambiar constantemente por la alteración del entorno de negocio (leyes, competidores), por el descubrimiento de nuevos *stakeholders*, o porque la comprensión del problema mejora a medida que se desarrolla el sistema.

La **Gestión de Requisitos** (*Requirements Management*) es el proceso de comprender y controlar estos cambios en los requisitos del sistema a lo largo del tiempo.

Los requisitos se dividen en:

- **Estables (Enduring):** Derivados de la actividad principal de la organización (ej. en un hospital, siempre habrá médicos y pacientes).
- **Volátiles (Volatile):** Requisitos propensos al cambio durante o después del desarrollo, dependientes de políticas gubernamentales o fluctuaciones del mercado.

Para gestionar este dinamismo, se exige la implementación de matrices de **Trazabilidad**, que registran los vínculos de dependencia matemática. Existen tres tipos fundamentales de trazabilidad:

1. **Trazabilidad de origen (Source):** Vincula un requisito específico con el *stakeholder* o documento normativo que lo originó.
2. **Trazabilidad de requisitos (Requirements):** Mapea las dependencias lógicas internas entre diferentes requisitos dentro del mismo documento.
3. **Trazabilidad de diseño (Design):** Vincula cada requisito funcional con los módulos de software y fragmentos de código arquitectónico que lo implementan.

4.6. El Documento SRS (Software Requirements Specification)

El SRS es el artefacto final de la fase de análisis. Es la declaración oficial de lo que los desarrolladores deben implementar. Declara exhaustivamente *qué* debe hacer el sistema, no *cómo* hacerlo arquitectónicamente.

El estándar **IEEE 830** propone la siguiente tabla de contenidos estructural:

1. **Prefacio e Introducción:** Propósito del sistema, alcance y **Glosario** (crítico para evitar ambigüedades semánticas).
2. **Requisitos de Usuario:** Descritos para el cliente en lenguaje natural.
3. **Arquitectura de Sistema:** Una visión macroscópica de alto nivel.
4. **Especificación de Requisitos de Sistema:** Requisitos funcionales y no funcionales detallados matemáticamente (FR y NFR).
5. **Modelos de Sistema:** Diagramas UML iniciales (Casos de Uso, diagramas de estado).
6. **Evolución y Apéndices:** Previsión de futuros cambios e índices de trazabilidad.

Observación 4.3 (La Perspectiva Ágil). En los sectores críticos (médico, aeroespacial, ferroviario), el documento SRS formal es **obligatorio por ley** para la certificación de seguridad. En metodologías ágiles (Scrum), la filosofía cambia: no se abandona la especificación, sino que se fragmenta. El SRS monolítico se sustituye por el **Product Backlog**, y los requisitos cambian de forma a *User Stories* vinculadas a *Tests de Aceptación*.

Ejemplo 4.4 (Clasificación y Verificabilidad de Requisitos). **Contexto:** Estás analizando el sistema de un dron de vigilancia autónomo. Te proporcionan la siguiente lista cruda de requerimientos redactados por el cliente.

Lista Cruda:

- a) “El dron debe volar de forma segura.”
- b) “El sistema debe transmitir el flujo de vídeo a la base terrestre cifrado bajo el estándar AES-256.”
- c) “El sistema debe permitir al operador ordenar el retorno a la base mediante la pulsación de un único botón.”
- d) “El sistema debe calcular su posición basándose en las leyes del movimiento uniformemente acelerado si se pierde la señal GPS.”

Problema: Clasifica cada requisito (FR, NFR, Dominio) y determina si es *verificable*. Si no lo es, reescríbelo formalmente.

Resolución Analítica:

- Req. a):

- *Clasificación:* Requisito No Funcional (Fiabilidad/Seguridad).
- *Evaluación:* **No Verificable**. El término “forma segura” es subjetivo y no se puede escribir un test binario para él.
- *Reescritura:* “El dron debe incluir un sistema de evasión de obstáculos que detecte objetos a una distancia mínima de 5 metros y desvíe su trayectoria en menos de 0.5 segundos.”

- Req. b):

- *Clasificación:* Requisito No Funcional (Seguridad/Estándares).

- *Evaluación:* **Verificable**. Es comprobable matemáticamente si el algoritmo de cifrado aplicado al flujo de bytes corresponde al estándar AES-256.
- **Req. c):**
 - *Clasificación:* Requisito Funcional (FR). Interacción de usuario.
 - *Evaluación:* **Verificable**. Se puede probar empíricamente pulsando el botón y verificando el cambio de estado del dron.
- **Req. d):**
 - *Clasificación:* Requisito de Dominio (y Funcional implícito).
 - *Evaluación:* **Verificable**. Se puede inyectar una pérdida de señal GPS en un entorno de simulación y comparar la coordenada calculada por el dron con el cálculo matemático teórico.

Capítulo 5

El Lenguaje de Modelado UML

5.1. Fundamentos de Modelado: Modelos, Vistas y Notaciones

Antes de introducir cualquier lenguaje específico, es fundamental establecer una base epistemológica rigurosa sobre qué significa “modelar” en ingeniería del software, basándonos en los conceptos formales de Martin Fowler.

En ingeniería, la abstracción es la herramienta principal para dominar la complejidad matemática y estructural de un sistema. Esta abstracción se materializa a través de tres conceptos fuertemente acoplados pero semánticamente distintos:

- Definición 5.1** (Modelo, Vista y Notación). - **Modelo (La Semántica):** Es una representación simplificada de un sistema o de una parte de él. Contiene la verdad estructural y lógica subyacente. Su propósito es ayudar a comprender y comunicar la esencia del problema antes de implementarlo.
- **Vista (La Proyección):** Es una proyección parcial del modelo desde un punto de vista específico. Matemáticamente, si el Modelo es un espacio n -dimensional, una vista es una proyección ortogonal sobre un subespacio k -dimensional ($k < n$), mostrando solo lo relevante para un *stakeholder* específico (analista, desarrollador, tester).
 - **Notación (La Sintaxis):** Es el conjunto riguroso de reglas gráficas o textuales (el alfabeto y la gramática) utilizadas para representar visualmente el modelo y sus vistas.

Las vistas de un mismo sistema no son disjuntas; se superponen e integran. Ejemplos clásicos de vistas incluyen la *Vista Lógica* (estructura de clases), la *Vista de Procesos* (comportamiento en ejecución), la *Vista Física* (distribución en hardware) y la *Vista de Casos de Uso* (interacción del usuario).

5.2. Introducción a UML (Unified Modeling Language)

Durante los años 80 y 90, existía la “guerra de los métodos”: docenas de notaciones incompatibles para el diseño orientado a objetos. **UML** fue creado por los “Tres Amigos” (Grady Booch, Jim Rumbaugh e Ivar Jacobson) para unificar estos enfoques bajo el auspicio del OMG (Object Management Group).

Idea Clave: Naturaleza y Meta-modelo de UML

UML es un **lenguaje** de modelado visual de propósito general, **no es un proceso ni una metodología**. UML nos proporciona el vocabulario y las reglas sintácticas para dibujar los planos del software, pero no nos dice en qué orden debemos dibujarlos.

Además, UML está estrictamente definido por su propio meta-modelo interno estandarizado a través de la arquitectura **MOF (Meta-Object Facility)**, lo que garantiza que las herramientas interpreten los diagramas de la misma manera.

5.2.1. Los Tres Modos de Uso de UML (Martin Fowler)

La profesora hace especial hincapié en la teoría de Martin Fowler, quien define que UML puede aplicarse en un proyecto bajo tres niveles distintos de rigor:

1. **UML como Boceto (*Sketch*)**: Uso informal y ágil (típicamente en una pizarra, papel o herramienta sencilla) cuyo objetivo es comunicar ideas complejas rápidamente, explorar alternativas o explicar un bloque de código.
2. **UML como Plano (*Blueprint*)**: Uso detallado y exhaustivo utilizando herramientas CASE especializadas. Se emplea para describir el sistema completo *antes* de programar (*Forward Engineering*) o para documentarlo a partir de un sistema ya existente (*Reverse Engineering*).
3. **UML como Lenguaje de Programación (*Programming Language*)**: El nivel más extremo de abstracción, asociado a la *Model Driven Architecture* (MDA). Aquí, el modelo UML es ejecutable por sí mismo, y la totalidad del código fuente de producción se compila o autogenera directamente desde los diagramas, sin intervención manual de programación.

5.3. Los Bloques de Construcción de UML

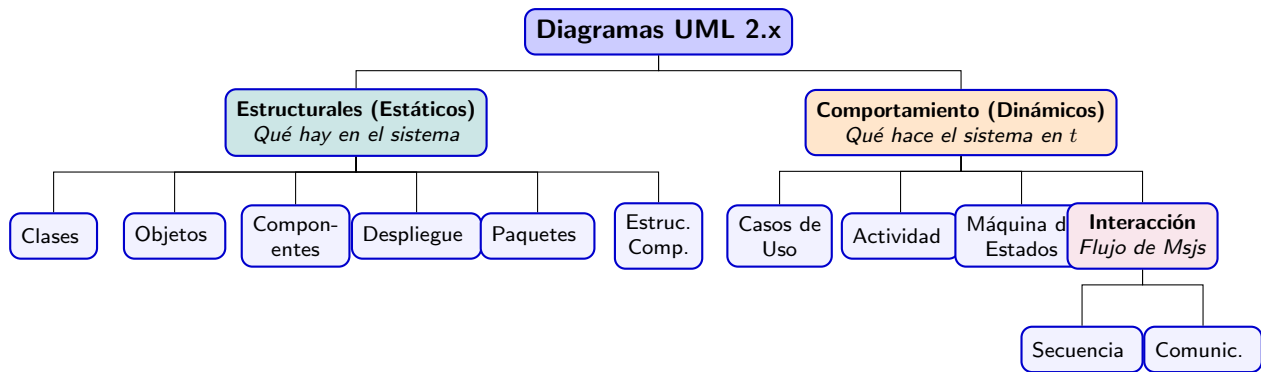
El vocabulario de UML se construye mediante la combinación ortogonal de tres bloques fundamentales: Elementos, Relaciones y Diagramas.

- **Elementos (*Things*)**: Son los sujetos y objetos del modelo. Se dividen en:
 - *Estructurales*: Las partes estáticas del modelo (Clases, Interfaces, Componentes, Nodos).
 - *De Comportamiento*: Las partes dinámicas (Interacciones, Máquinas de Estado).
 - *De Agrupación*: Para organizar el modelo (Paquetes).
 - *De Anotación*: Para añadir información semántica (Notas).
- **Relaciones (*Relationships*)**: La semántica que une a los elementos. Las cuatro principales son: Dependencia, Asociación, Generalización (Herencia) y Realización (Interfaces).
- **Diagramas (*Diagrams*)**: Son las proyecciones visuales de grafos compuestos por elementos (nodos) y relaciones (aristas).

5.4. Taxonomía de los Diagramas UML 2.x

UML clasifica sus diagramas de forma dicotómica basándose en si representan la arquitectura estática o el comportamiento dinámico dependiente del tiempo.

Existen 14 tipos de diagramas oficiales en UML 2.x, clasificados matemáticamente en dos grandes conjuntos disjuntos que particionan el espacio de modelado: **Estructurales** y **de Comportamiento** (dentro de los cuales existe el subconjunto de **Interacción**).



5.4.1. Diagramas Estructurales

Ignoran la variable tiempo (t). Muestran los elementos del sistema y sus relaciones invariantes topológicas.

- **Diagrama de Clases:** El más común. Muestra la estructura estática del sistema en términos de clases, interfaces, atributos, operaciones y sus relaciones formales (asociación, herencia, composición).
- **Diagrama de Objetos:** Una “fotografía” instantánea de la memoria en un t_0 , mostrando las instancias reales de las clases y sus valores.
- **Diagrama de Componentes:** Muestra la organización estructural de los componentes modulares del software y sus dependencias (interfaces requeridas/ofrecidas).
- **Diagrama de Despliegue (Deployment):** Modela la topología del hardware físico (nodos) sobre el cual se ejecuta el software.

5.4.2. Diagramas de Comportamiento e Interacción

Modelan el comportamiento dinámico; son funciones donde el tiempo y los eventos son las variables independientes.

- **Diagrama de Casos de Uso:** Describe gráficamente las interacciones entre los actores externos (humanos u otros sistemas) y el sistema para lograr un objetivo.
- **Diagrama de Secuencia (Interacción):** Representa el intercambio cronológico de mensajes entre objetos a lo largo del tiempo. Las líneas de vida (*lifelines*) descienden verticalmente.
- **Diagrama de Actividades:** Modela el flujo de control o de datos mediante grafos dirigidos (nodos de acción, decisiones lógicas y bifurcaciones concurrentes), análogo a un diagrama de flujo avanzado.
- **Diagrama de Máquina de Estados:** Modela los estados mutuamente excluyentes que puede adoptar un objeto durante su vida y las transiciones generadas por eventos matemáticos discretos.

5.5. Sintaxis Extendida y Reglas Metodológicas

Para mantener el estándar ágil sin perder expresividad, UML permite ampliar su sintaxis base e incorpora normativas fundamentales de abstracción.

5.5.1. Mecanismos de Extensión

Existen mecanismos estándar para añadir semántica a un modelo UML sin necesidad de modificar su metamodelo formal:

- **Estereotipos (*Stereotypes*)**: Permiten crear nuevos tipos de bloques de construcción. Se denotan utilizando guillemets (*comillas angulares*), por ejemplo: <<interface>>, <<control>> o <<actor>>.
- **Valores Etiquetados (*Tagged Values*)**: Permiten adjuntar propiedades clave-valor a los elementos del modelo. Se colocan entre llaves, por ejemplo: {author="Fowler"} o {version=1.2}.
- **Restricciones (*Constraints*) y OCL**: Reglas o condiciones lógicas que el elemento debe cumplir obligatoriamente. Suelen delimitarse por llaves {edad >= 18}. Para evitar la ambigüedad del lenguaje natural, se utiliza un lenguaje matemático estandarizado llamado **OCL (*Object Constraint Language*)**.

Idea Clave: La Regla de la Información Suprimida

Una de las reglas metodológicas más importantes del diseño UML dicta que **la ausencia de un elemento en un diagrama no implica de ninguna manera que no exista en el sistema real**.

Su omisión indica simplemente que:

1. No es relevante para la *vista* o el propósito actual del diagrama.
2. O bien, aún no ha sido especificado.

En la práctica, un ingeniero nunca debe intentar modelar el 100 % del sistema en un único diagrama (anti-patrón de "Parálisis por Análisis"), ya que destruye su propósito comunicativo.

5.6. Herramientas CASE de Modelado: Visual Paradigm

Como hemos visto en el Capítulo 1, las herramientas *Upper-CASE* automatizan el proceso de diseño. Para este curso (y en la industria formal) se utiliza **Visual Paradigm**.

Observación 5.2 (El valor de una herramienta CASE y el Round-Trip Engineering). Dibujar UML en papel o en software genérico (como Paint o PowerPoint) produce diagramas "muertos". Una herramienta CASE genuina genera un **Modelo de Dominio Subyacente** vivo.

Si renombras una clase en un Diagrama de Clases en Visual Paradigm, ese cambio se propaga y actualiza automáticamente al Diagrama de Secuencia donde participe dicha clase, manteniendo la coherencia. Además, permiten el **Round-Trip Engineering** (Ingeniería de Ciclo Completo), que es la suma de:

- *Forward Engineering*: Generación automática de código esqueleto Java/C++ a partir del diagrama.
- *Reverse Engineering*: Generación del diagrama UML a partir de código fuente existente.

Capítulo 6

Modelado Funcional: Casos de Uso

6.1. Introducción al Modelado Funcional

El modelado funcional es la fase en la que transcribimos los requisitos elicitados a un modelo formal que define el comportamiento del sistema desde una perspectiva externa.

En UML, el instrumento principal para esta tarea es el **Modelo de Casos de Uso** (*Use Case Model*).

Definición 6.1. Un **Caso de Uso** describe una secuencia de interacciones entre un actor (entidad externa) y el sistema, orientada a lograr un objetivo observable y de valor para el actor.

Idea Clave: El Sistema como Caja Negra

El modelado de casos de uso responde *exclusivamente* a la pregunta “¿Qué hace el sistema?”. En esta fase, el sistema se trata matemáticamente como una **caja negra**: nos interesan las entradas (estímulos de los actores) y las salidas (resultados funcionales), ignorando por completo *cómo* se implementan estructuralmente por dentro.

6.2. Elementos Sintácticos del Diagrama

La sintaxis del diagrama de casos de uso es intencionalmente minimalista, componiéndose de tres primitivas topológicas principales:

1. **Actores:** Entidades externas que interactúan con el sistema. Se representan mediante un *stick-man*.
 - **Crucial:** Un actor representa un **rol**, no a una persona física. Una misma persona puede interpretar varios roles (ej. un *Profesor* que también actúa como *Estudiante* en otro curso).
 - Pueden ser humanos, dispositivos de hardware externos (ej. *Sensor de Temperatura*), u otros sistemas de software (ej. *Sistema Bancario*).
 - **Actor Principal:** Inicia el caso de uso para lograr su objetivo.
 - **Actor Secundario (de Soporte):** El sistema acude a él durante la ejecución del caso de uso para obtener un servicio (ej. un servidor de validación de tarjetas de crédito).
2. **Casos de Uso:** Representan los objetivos o funciones. Se dibujan como elipses con el nombre de la funcionalidad en su interior (típicamente en formato Verbo + Objeto, ej. “Comprar Billete”).
3. **Frontera del Sistema (*System Boundary / Subject*):** Un rectángulo que engloba todos los casos de uso. **Define formalmente el límite del proyecto:** todo lo que está dentro de esta frontera es lo que deberá ser diseñado, codificado, verificado y validado, constituyendo el producto final de software (los actores siempre quedan fuera).

6.3. Relaciones en UML para Casos de Uso

El poder expresivo del diagrama radica en las relaciones semánticas entre sus elementos. Estas se rigen por reglas matemáticas estrictas.

6.3.1. Relaciones Actor - Caso de Uso

La única relación permitida entre un actor y un caso de uso es la **Asociación**, representada por una línea continua sin flechas (o con flecha indicando la dirección del estímulo inicial). Significa que el actor y el sistema se comunican en el contexto de ese caso de uso.

6.3.2. Relaciones Actor - Actor

La única relación permitida entre actores es la **Generalización** (Herencia), representada por una línea continua terminada en un triángulo hueco. Si el Actor *A* hereda del Actor *B*, *A* asume el rol de *B* y puede interactuar con todos los casos de uso asociados a *B*, además de los suyos propios.

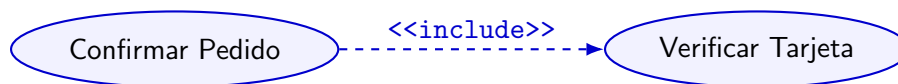
6.3.3. Relaciones entre Casos de Uso

Para evitar la redundancia y modelar comportamientos complejos, UML permite relacionar Casos de Uso entre sí. Es crucial dominar la diferencia lógica entre `<<include>>` y `<<extend>>`.

Relación de Inclusión (`<<include>>`)

Representa un comportamiento que es **incondicional** y **obligatorio**. Si el caso de uso base *A* incluye a *B*, la ejecución de *A* implica matemáticamente la ejecución garantizada de *B*. Se utiliza para factorizar comportamientos comunes a varios casos de uso.

Notación: Flecha discontinua desde el caso base *A* hacia el caso incluido *B*.



Relación de Extensión (`<<extend>>`)

Representa un comportamiento **condicional** y **opcional**. El caso de uso *B* extiende al caso base *A* solo si se cumple una condición lógica específica en un "punto de extensión" (*Extension Point*). Se utiliza para modelar el manejo de errores, excepciones o caminos alternativos raros.

Notación: Flecha discontinua desde el caso extensor *B* **hacia** el caso base *A*. ¡Ojo con la dirección inversa!



6.4. La Especificación Textual: El Verdadero Caso de Uso

Observación 6.2 (La Trampa del Diagrama). La profesora enfatiza fuertemente en las diapositivas que **los detalles sobre el comportamiento del sistema no se encuentran en el diagrama, sino en las descripciones textuales**. El diagrama por sí solo no sirve como especificación; podría incluso llegar a omitirse, pero la descripción textual jamás.

Un Modelo de Casos de Uso riguroso consta del diagrama *y* de un documento textual para cada burbuja. Esta descripción formal debe contener:

- **Precondiciones:** El estado matemático en el que debe estar el sistema *antes* de iniciar el caso de uso.
- **Escenario Principal de Éxito (*Happy Path*):** La secuencia de pasos ideal donde todo funciona correctamente y el actor logra su objetivo sin errores.
- **Escenarios Alternativos y Excepciones:** Qué ocurre si algo falla (ej. el usuario introduce una contraseña incorrecta o se corta la conexión al servidor de pago). Estos pasos suelen ser los que dan origen a las relaciones <<extend>>.
- **Postcondiciones:** El estado garantizado del sistema una vez finalizado el caso de uso.

6.5. Trazabilidad: Requisitos vs. Casos de Uso

Es un error conceptual asumir una biyección (relación 1 a 1) entre Requisitos Funcionales y Casos de Uso. La relación es de muchos a muchos ($N : M$):

- Un único Caso de Uso puede satisfacer múltiples Requisitos Funcionales simultáneamente.
- Un único Requisito Funcional complejo puede requerir ser desglosado en varios Casos de Uso.
- **Funciones Internas:** No todos los requisitos funcionales dan origen a un Caso de Uso. Las funciones automáticas internas del sistema, que ocurren sin la intervención de un actor, no se modelan como casos de uso.
- **Requisitos No Funcionales en C.U.:** Un Caso de Uso específico puede llevar asociados Requisitos No Funcionales (por ejemplo, requerimientos estrictos de seguridad o rendimiento para esa transacción concreta).

Idea Clave: Diferencia Estricta de Perspectiva

Aunque están ligados, responden a enfoques distintos:

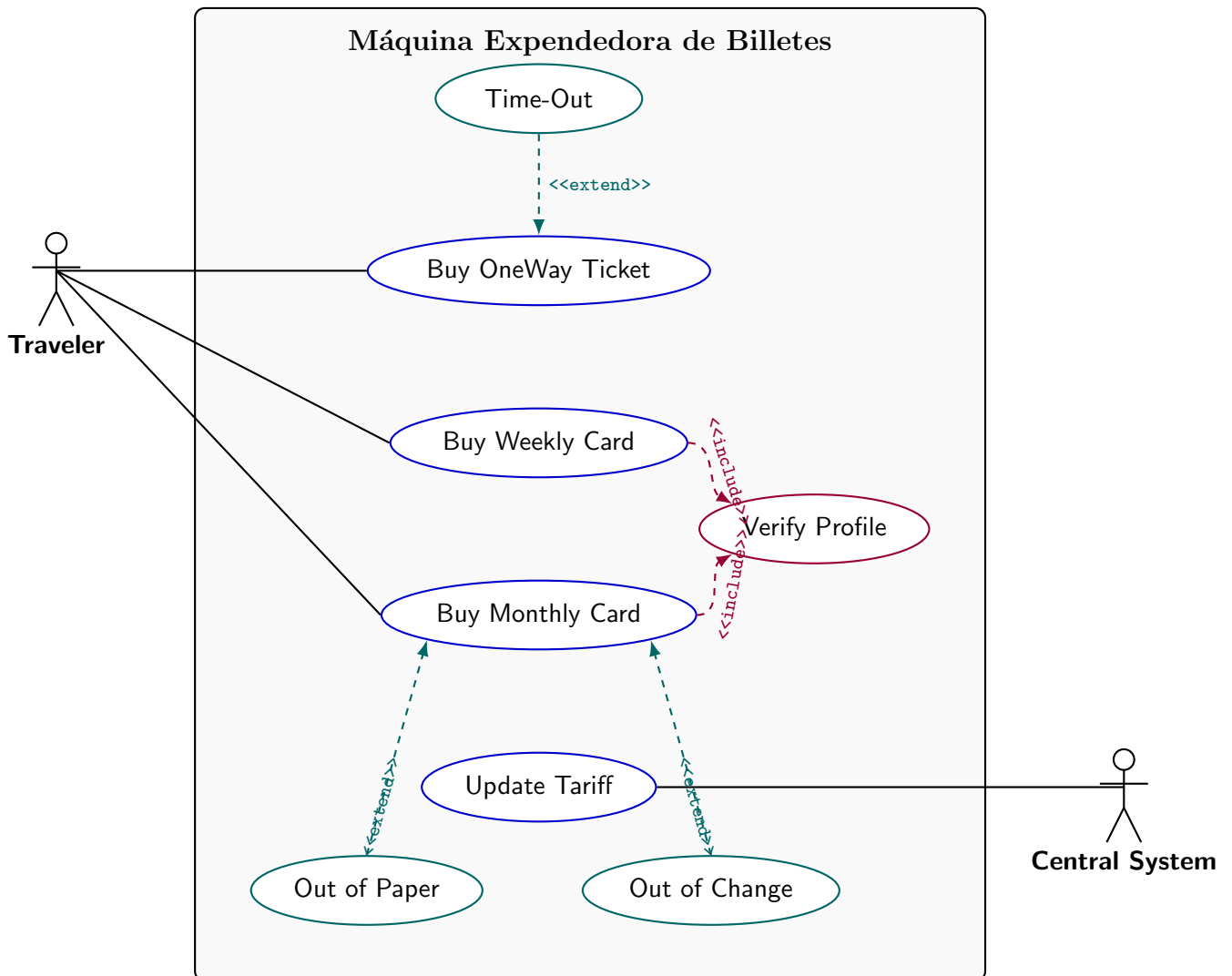
- El **Requisito Funcional** describe *qué debe hacer el sistema* internamente.
- El **Caso de Uso** describe *la interacción* externa entre el actor y el sistema.

Para garantizar la completitud del análisis en ingeniería, se construye una **Matriz de Trazabilidad** donde se mapea qué UC cubre qué REQ, garantizando matemáticamente que no queden requisitos huérfanos.

Ejemplo 6.3 (Ejercicio de Examen: Máquina Expendidora de Billetes). **Contexto Operativo:** Se requiere modelar el software de una máquina automática de billetes de tren.

- El *Viajero* interactúa con la máquina para comprar billetes sencillos, abonos semanales y abonos mensuales.
- Comprar un abono mensual o semanal requiere un paso común obligatorio: verificar el perfil del viajero.
- Durante cualquier proceso de compra de billete, si el viajero tarda demasiado, puede ocurrir un *Time-Out*. Si la máquina se queda sin papel, debe dispararse una excepción de *Fuera de Papel*. Si se queda sin monedas, una de *Fuera de Cambio*. Estas son interrupciones condicionales del flujo normal.
- Paralelamente, el *Sistema Informático Central* (un sistema externo) debe poder conectarse a la máquina para *Actualizar la Tarifa*.

Modelado UML (Solución Gráfica Vectorial):



Observación 6.4 (Análisis del Ejercicio). Nota cómo el actor **Traveler** se comunica directamente con las compras. Las excepciones como *Time-Out* no tienen una línea continua hacia el actor, ya que son

anomalías que extiende el sistema internamente a través de `<<extend>>`. Por su parte, la verificación de perfil es obligatoria para abonos, de ahí el uso del `<<include>>` apuntando desde la compra hacia la verificación. Además, el **Central System** actúa como un actor porque es un sistema informático externo que interactúa con las fronteras de la máquina para modificar la tarifa.

Capítulo 7

Modelado Estructural: Diagrama de Clases

7.1. Fundamentos del Diagrama de Clases

El **Diagrama de Clases (CD)** es el pilar central del modelado orientado a objetos en UML. Es un diagrama estructural (estático) que describe la topología del sistema mostrando sus clases, sus atributos, sus operaciones y las relaciones matemáticas formales que existen entre ellas.

Idea Clave: Perspectivas de Modelado

Según Fowler, un Diagrama de Clases puede y debe dibujarse iterativamente desde tres niveles de abstracción (perspectivas):

1. **Conceptual (Modelo de Dominio):** Modela los conceptos del problema en el mundo real. **Regla Crítica:** En esta fase NO se modelan métodos de software. En su lugar, se definen las **Responsabilidades** (el conjunto de tareas que la clase debe garantizar lógicamente).
2. **Especificación (Diseño):** Modela interfaces y tipos abstractos. Se enfoca en *qué* hace el software y cómo interactúan sus partes, desvinculado del lenguaje de programación.
3. **Implementación (Modelo de Sistema):** Modela las clases exactas que se programarán en Java/C++, incluyendo navegación explícita, tipos de datos precisos y visibilidad.

7.2. Anatomía Precisa de una Clase en UML

Una clase en UML se representa mediante un rectángulo dividido rígidamente en tres compartimentos. Ninguno es obligatorio salvo el nombre, pero el orden topológico es estricto.

Observación 7.1 (Convenciones Topográficas de Nomenclatura). Para asegurar la consistencia del modelo, UML estipula normativas de escritura:

- El **Nombre de la Clase** debe ir siempre en UpperCamelCase (ej. CuentaBancaria).
- Los **Atributos y Operaciones** deben ir en lowerCamelCase (ej. saldoActual, calcularInteres()).

NombreDeLaClase
Atributos
<i>v</i> nombre : Tipo [mult] = valor
Operaciones
<i>v</i> metodo(arg : Tipo) : TipoRetorno

7.2.1. Visibilidad (*v*) y la Firma de la Operación

La encapsulación es crítica en ingeniería. UML define cuatro modificadores de visibilidad ortogonales:

- + (**Public**): Accesible por cualquier elemento del sistema.
- - (**Private**): Accesible únicamente dentro de la propia clase.
- # (**Protected**): Accesible por la clase y sus subclases derivadas.
- ~ (**Package**): Accesible por cualquier clase dentro del mismo paquete (espacio de nombres).

Además, en el compartimento de operaciones, lo que se modela es la **Firma** (*Signature*) del método (nombre + lista ordenada de parámetros). Esto permite modelar en UML la **Sobrecarga** (*Overloading*) de métodos, declarando el mismo nombre varias veces pero con firmas matemáticamente distintas.

7.2.2. Reglas Críticas de Atributos y Operaciones

Las diapositivas de la profesora dictan reglas estrictas para evitar modelos semánticamente incorrectos:

1. **No confundir Atributos con Clases:** Si un atributo tiene un comportamiento complejo o una identidad propia (ej. una *Direccion* que no es un simple *String* sino que tiene calle, CP y ciudad), debe modelarse como una **Clase separada**, no como un atributo primitivo.
2. **Pluralidad es Multiplicidad:** Es un error de diseño modelar arreglos como atributos plurales (ej. - telefonos : *String*). La forma correcta de modelar una colección es especificando la multiplicidad algorítmica: - telefono : *String* [1..*].
3. **Atributos Derivados (/):** Si el valor de un atributo se calcula matemáticamente a partir de otros (ej. la *edad* a partir de la *fecha de nacimiento*), su nombre debe ir precedido por una barra (/ edad : int).

7.2.3. Ejemplo de Clase a Nivel de Implementación

Estudiante
- matricula : String [1]
- /notaMedia : float = 0.0
- telefono : String [0..*]
+ matricular(asig : String) : boolean
+ getNotaMedia() : float
calcularExpediente() : void

7.3. Taxonomía Exacta de las Relaciones UML

Las relaciones definen cómo las clases interactúan y comparten datos. Cada línea en UML tiene un significado estricto en la gestión de memoria y el acoplamiento del software.

7.3.1. Asociación (Association)

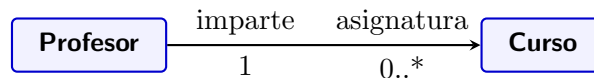
Representa una conexión estructural semántica entre dos clases. Implica que un objeto de una clase “conoce” o “tiene una referencia persistente” a un objeto de la otra clase. Se dibuja con una línea sólida.

Observación 7.2 (Asociaciones y Persistencia). Una de las heurísticas clave enseñadas por la profesora es que **las asociaciones estructurales normalmente implican persistencia**. Es decir, si existe una Asociación entre *Cliente* y *Pedido*, esta relación debe ser guardada en la Base de Datos. Si la conexión es efímera y no requiere ser guardada, probablemente deba modelarse como una Dependencia.

Navegabilidad y Dependencia Circular:

- Si hay una flecha abierta ($\rightarrow$), la asociación es *unidireccional* (la clase A conoce a B y almacena un puntero hacia ella, pero B no conoce a A). **Esta es la forma preferida en el diseño orientado a objetos**, ya que reduce el acoplamiento.
- Si no hay flechas, es *bidireccional* (A conoce a B y B conoce a A). Esto crea un acoplamiento cíclico muy fuerte y debe usarse con extrema precaución.

Nombres de Rol (*Role Names*): Los extremos de la línea de asociación pueden tener etiquetas. Estas no designan el nombre de la clase, sino la **función (rol)** que cumple una clase para la otra.



Multiplicidad (Cardinalidad): Define matemáticamente cuántas instancias de la clase destino pueden estar vinculadas a **UNA** instancia de la clase origen.

- 1 : Exactamente uno (obligatorio).
- 0..1 : Cero o uno (opcional).
- * ó 0..* : Cero o más (una colección, lista o conjunto).
- 1..* : Al menos uno, hasta infinito.

Restricciones en Asociaciones ({xor})

UML permite usar el lenguaje formal **OCL (*Object Constraint Language*)** para añadir restricciones lógicas. Una muy común en exámenes es la restricción {xor}, utilizada cuando una clase origen puede tener una asociación con la clase B **O** con la clase C, *pero nunca con ambas simultáneamente*. Se representa con una línea discontinua que une ambas asociaciones etiquetada con {xor}.

Asociación Reflexiva (Unaria)

Ocurre cuando una clase tiene una relación de asociación consigo misma. Es vital utilizar los nombres de rol para distinguir los extremos lógicos. Por ejemplo, en una clase *Empleado*, la relación puede tener el rol *jefe* [1] en un extremo y *subordinados* [0..*] en el otro.

7.3.2. Clases Asociativas (Association Classes)

Existen situaciones donde una relación de asociación $N : M$ entre dos clases contiene **información propia** que no pertenece lógicamente a ninguna de las dos clases extremas.

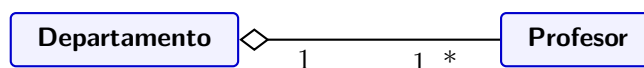
La profesora pone el ejemplo de *Impiegato* (Empleado) y *Progetto* (Proyecto). Un empleado trabaja en varios proyectos y un proyecto tiene varios empleados. La relación “LavoraPresso” (Trabaja en) tiene un atributo propio: las *horas de trabajo*. Este atributo no pertenece al empleado ni al proyecto por sí solo, pertenece a la **intersección** de ambos.

La sintaxis UML es una clase unida por una línea discontinua a la línea de asociación principal.

7.3.3. Agregación (Aggregation)

Es una variante de la asociación que modela una relación “**Todo - Parte**”. La clave matemática es que **los ciclos de vida son independientes**. Si el Todo se destruye, la Parte sigue existiendo en memoria.

Sintaxis UML: Rombo **HUECO** en el lado del “Todo”.

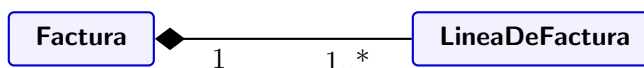


Interpretación: Un Departamento agrupa a varios Profesores. Si el Departamento universitario cierra (se elimina el objeto de la BD), los Profesores no mueren; continúan existiendo y son reasignados a otro lugar.

7.3.4. Composición (Composition)

Es la forma más fuerte y restrictiva de agregación. Modela una relación “**Todo - Parte**” estricta donde **los ciclos de vida son estrictamente coincidentes**. La Parte no puede existir sin el Todo. Si se elimina el Todo, la memoria de todas sus Partes se libera en cascada.

Sintaxis UML: Rombo **RELLENO** en el lado del “Todo”.



Interpretación: Una Factura se compone inexorablemente de Líneas de Factura. Una Línea carece de sentido lógico y matemático si no pertenece a una Factura concreta. Si destruyes el objeto Factura, sus Líneas deben borrarse implícitamente.

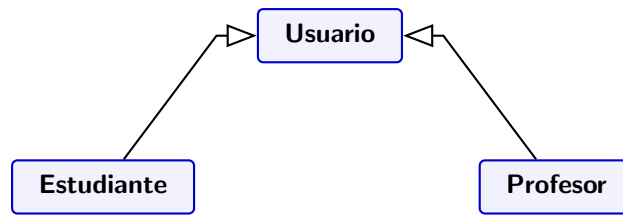
7.3.5. Generalización (Herencia)

Modela la relación taxonómica “**Es-un**” (*Is-a*). Representa la herencia orientada a objetos clásica donde una Subclase hereda la estructura (atributos) y el comportamiento (operaciones) de una Superclase genérica.

Sintaxis UML: Flecha con un triángulo **HUECO Y CERRADO** apuntando siempre hacia la Superclase.

Idea Clave: Sustituibilidad y Herencia Múltiple

El principio lógico subyacente es la **Sustituibilidad (Polimorfismo)**: En cualquier lugar del software donde se espere la Superclase, el sistema debe admitir una Subclase de forma transparente. Además, UML, al ser agnóstico del lenguaje, **soporta Herencia Múltiple** (una clase derivada de varias superclases), aunque lenguajes topológicos puros como Java o C# no lo permitan directamente en su sintaxis de clases.



Observación 7.3 (Estrategias de Identificación: Bottom-up vs. Top-down). La profesora indica que existen dos enfoques formales para diseñar jerarquías de generalización:

- **Bottom-up:** Se identifican primero las clases concretas. Al notar que comparten atributos idénticos, se agrupan abstrayéndolas en una nueva Superclase común.
- **Top-down:** Se define primero la clase más general y teórica, y luego se especializa iterativamente creando subclases más concretas.

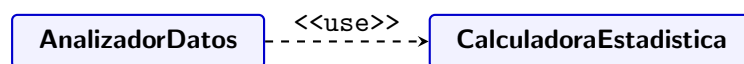
7.3.6. Dependencia (Dependency)

Es la relación más débil, efímera e inestable de UML. Indica que un cambio en el elemento independiente afectará semánticamente al elemento dependiente. Suele indicar que una clase A “usa” a una clase B de forma temporal (ej. B es un parámetro de un método, o A insta a B internamente).

No hay un vínculo de memoria persistente (*puntero*) como en la Asociación.

Sintaxis UML: Línea **DISCONTINUA** con una flecha abierta (->). Para clarificar su naturaleza, UML estandariza los siguientes estereotipos principales:

- **<<use>>:** El elemento requiere la presencia del otro para su definición o implementación.
- **<<call>>:** El elemento origen invoca una operación del elemento destino.
- **<<create>> o <<instantiate>>:** El origen actúa como fábrica y crea instancias del destino.



7.4. Conceptos Avanzados de Modelado Estático

El Capítulo 9 y las clases complementarias introducen piezas arquitectónicas fundamentales para mapear el código real en UML.

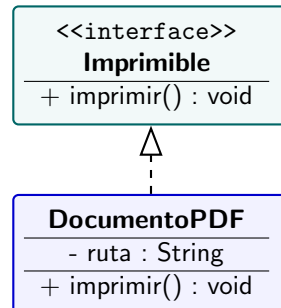
7.4.1. Clases Abstractas e Interfaces

La segregación de interfaces es un principio vital del diseño OO. UML formaliza esto de la siguiente manera:

- **Clase Abstracta:** Una clase que no puede ser instanciada porque es deliberadamente incompleta (tiene al menos un método declarado pero sin implementar). Define una plantilla estructural común para una jerarquía. En UML, tanto el nombre de la clase como los métodos abstractos **se escriben obligatoriamente en *Cursiva (Italics)***.
- **Interfaz:** Un contrato de comportamiento puramente formal. Carece de estado interno (no tiene atributos instanciables) y todos sus métodos carecen de cuerpo. En UML, se etiqueta con el estereotipo **<<interface>>**.

La relación topológica por la cual una clase concreta firma el contrato de una interfaz se denomina **Realización** (*Realization*).

Sintaxis UML: Flecha **DISCONTINUA** terminada en un triángulo hueco apuntando hacia la interfaz (es visualmente una mezcla entre Generalización y Dependencia).

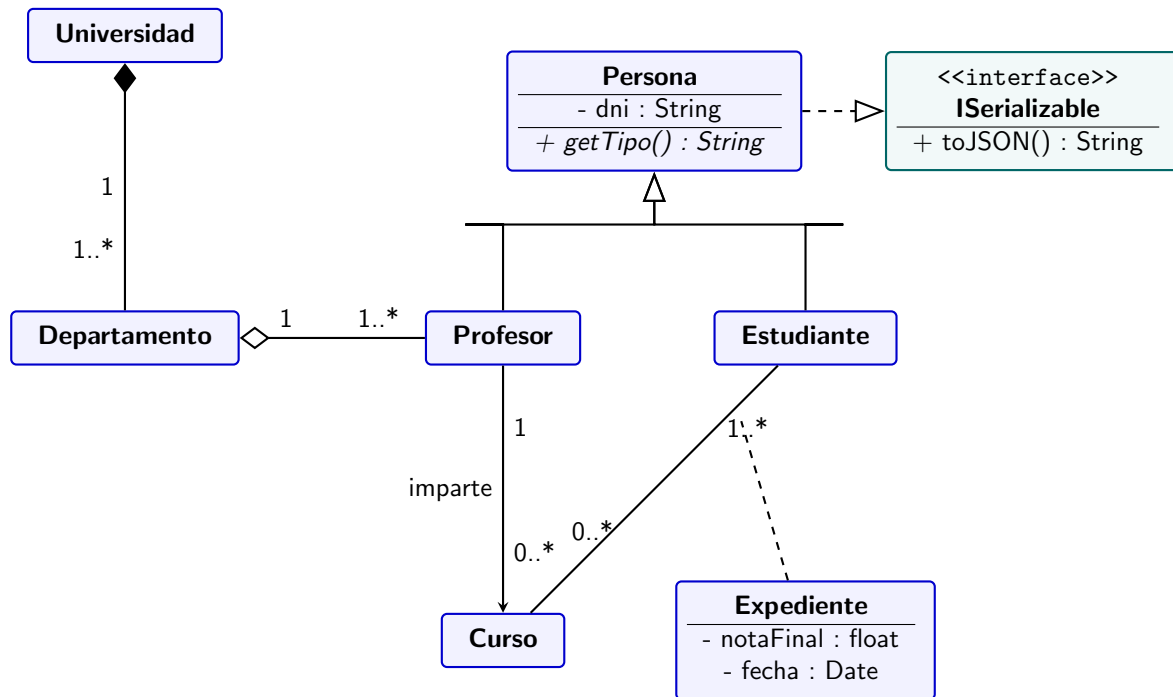


7.4.2. Clases Activas, Paramétricas y Enumeraciones

- **Clases Activas (*Active Classes*):** Fundamental en Sistemas Operativos y concurrencia. Son clases cuyos objetos poseen su propio hilo de control (*thread*). En UML se dibujan con líneas laterales verticales **dobles**.
- **Clases Paramétricas (Templates/Generics):** Utilizadas para programar tipos genéricos (como `ArrayList<T>` o `Map<K, V>`). En UML se denotan añadiendo un pequeño rectángulo con borde discontinuo en la esquina superior derecha de la clase que contiene la letra del parámetro (ej. T).
- **Tipos Enumerativos (Enum):** Un conjunto finito de valores estáticos y predefinidos (ej. los días de la semana). Se modela usando una clase con el estereotipo <<enumeration>>.

7.5. Diagrama Integrador Resuelto: Gestión Universitaria

Ejemplo 7.4 (Modelado de Alto Nivel DGIM). **Enunciado:** Diseñar el diagrama de clases para el sistema universitario. Reglas de negocio: Una Universidad se compone de Departamentos. Los Departamentos agrupan a Profesores. Un Profesor hereda de Persona y puede impartir múltiples Cursos. Los Estudiantes (que también son Personas) se matriculan en Cursos a través de una clase de asociación llamada Expediente, que guarda la nota. Todos los objetos deben poder serializarse, para lo cual implementan la interfaz `ISerializable`.



Análisis de Precisión Matemática del Diagrama:

1. La composición entre **Universidad** y **Departamento** asegura que si la entidad Universidad se borra de la Base de Datos, sus Departamentos también sufren borrado en cascada.
2. **Persona** es una superclase abstracta (fuente cursiva). Su método `getTipo()` está en cursiva, obligando algorítmicamente a **Estudiante** y **Profesor** a proveer su propia implementación interna.
3. **Expediente** se modela como una **Clase Asociativa** ya que resuelve la complejidad topológica de una relación $N : M$ entre Estudiantes y Cursos, encapsulando las propiedades emergentes de la relación (`notaFinal`).
4. La realización de la interfaz **ISerializable** en la raíz de la jerarquía (**Persona**) garantiza matemáticamente que todas sus subclases especializadas heredarán el contrato y estarán forzadas a programar el método `toJSON()`.

Capítulo 8

Modelado Estructural Avanzado

8.1. Introducción a la Macroarquitectura

El Diagrama de Clases nos proporciona una topología precisa del nivel microscópico del software (el código). Sin embargo, a medida que la cardinalidad de clases $|C|$ crece, el diagrama pierde legibilidad. Para gobernar la complejidad sistémica, la ingeniería del software utiliza mecanismos de agrupamiento y abstracción arquitectónica.

UML proporciona herramientas matemáticas y visuales para modelar instancias, agrupar espacios de nombres, empaquetar módulos reemplazables y proyectar el software sobre el hardware físico.

8.2. Diagrama de Objetos (Object Diagram)

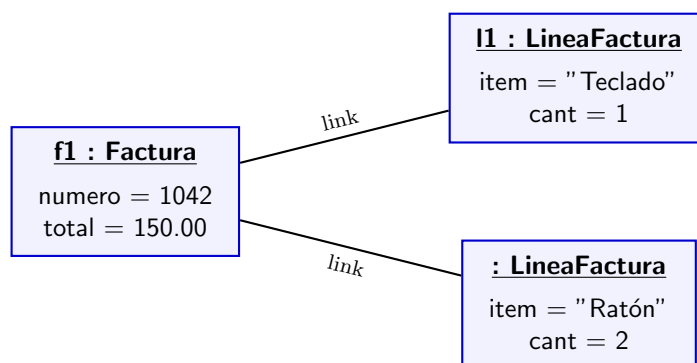
Mientras que un Diagrama de Clases representa las reglas invariantes y eternas del sistema, un **Diagrama de Objetos** es una función evaluada en un instante de tiempo t_0 . Modela una “fotografía” instantánea de la memoria RAM.

Idea Clave: Semántica del Diagrama de Objetos

Un diagrama de objetos es un grafo donde los nodos son **instancias** (objetos con estado/valores) y las aristas son **enlaces** (*links*, instanciaciones topológicas de las asociaciones entre clases).

Sintaxis rigurosa: El nombre de un objeto **siempre** debe ir subrayado y sigue el patrón explícito nombreInstancia : NombreClase.

Observación 8.1 (Objetos Anónimos). Si la identidad específica de la instancia no es matemáticamente relevante para la vista que se está modelando, UML permite omitir el nombre del objeto, dejando únicamente la clase subrayada (ej. : Factura). A esto se le conoce como un **objeto anónimo**.



8.3. Diagrama de Paquetes (Package Diagram)

Un **Paquete** es un mecanismo puramente lógico para organizar elementos (clases, interfaces, o incluso otros sub-paquetes anidados) en grupos semánticos. Es el equivalente topológico a un directorio o espacio de nombres (*namespace*). Su objetivo es minimizar el acoplamiento y maximizar la cohesión.

8.3.1. Nombres Cualificados e Identidad

La pertenencia a un paquete altera la identidad matemática de una clase. En UML, se utiliza el operador de resolución de ámbito `::` para definir el **Nombre Cualificado** (*Fully Qualified Name*).

Si una clase **Bibliotecario** reside dentro del sub-paquete **Utenti**, el cual está dentro de **Biblioteca**, su identidad absoluta en el sistema es **Biblioteca::Utenti::Bibliotecario**. Las clases del exterior deben usar esta ruta completa para instanciarla, mientras que las clases internas pueden usar el nombre relativo.

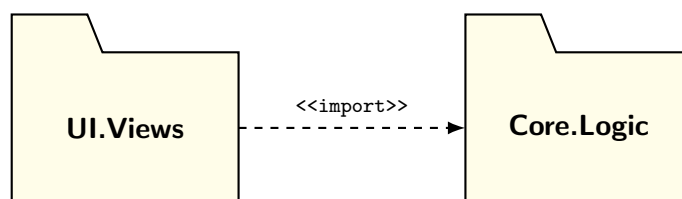
8.3.2. Reglas de Dependencia y Criterios de Agrupación

Las dependencias lógicas se modelan con flechas discontinuas. La profesora exige diferenciar estrictamente entre dos estereotipos de dependencia:

- `<<import>>`: Concede visibilidad **pública**. Los elementos del paquete importado se comportan como si hubieran sido definidos en el paquete importador (transitividad).
- `<<access>>`: Concede visibilidad **privada**. Los elementos accedidos están disponibles para el paquete origen, pero no se exponen al exterior si un tercer paquete importa al primero.

Idea Clave: Criterios de Empaquetado

¿Cómo sabemos qué clases meter en qué paquete? La heurística fundamental de diseño es la **Alta Cohesión**: se deben agrupar en el mismo paquete aquellas clases que colaboran íntimamente para realizar una responsabilidad o funcionalidad estrechamente correlacionada.



8.4. Diagrama de Componentes

Un **Componente** no es una clase. Es una unidad modular de software **autónoma**, **desplegable** e **intercambiable** que encapsula su estado y comportamiento detrás de interfaces estrictamente formales.

Definición 8.2 (Intercambiabilidad Estructural). Un componente C_1 puede ser sustituido analíticamente por un componente C_2 inalterando el sistema, si y solo si C_2 implementa el mismo conjunto de **Interfaces Ofrecidas** (*Provided Interfaces*) y requiere el mismo (o menor) subconjunto de **Interfaces Requeridas** (*Required Interfaces*) que C_1 .

Perspectivas de Modelado: La presentación de la profesora establece que un componente puede modelarse desde dos ópticas:

- **Black-box (Caja Negra):** Muestra exclusivamente el exterior del componente (sus puertos y las interfaces que ofrece y requiere). Oculta toda la complejidad interna.
- **White-box (Caja Blanca):** Penetra el componente para mostrar las clases, interfaces internas o sub-componentes que *realizan* (implementan) el comportamiento prometido.

Sintaxis Vectorial (Lollipop y Socket):

- **Lollipop (Círculo):** Interfaz ofrecida (lo que el componente expone).
- **Socket (Semicírculo):** Interfaz requerida (lo que el componente necesita).
- **Puertos (Ports):** Representados como pequeños cuadrados en el borde del componente, definen puntos formales de interacción.

8.5. Diagrama de Despliegue (Deployment Diagram)

El **Diagrama de Despliegue** mapea la topología de la arquitectura lógica (los componentes) sobre la arquitectura física del hardware. Modela **Nodos** (computadoras, servidores) y **Artefactos** (archivos ejecutables `.jar`, scripts).

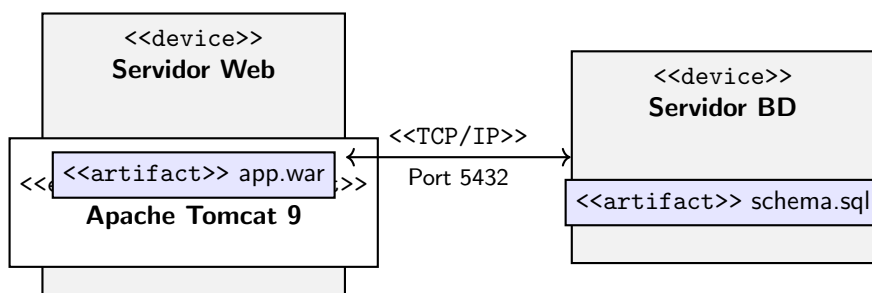
Al igual que los diagramas de clases y objetos, UML soporta dos niveles de abstracción topológica para el despliegue:

- **Nivel de Especificación (*Specification Level*):** Modela nodos conceptuales sin ligarlos a hardware específico (ej. un nodo genérico llamado “Servidor Web”).
- **Nivel de Instancia (*Instance Level*):** Modela las máquinas físicas exactas y sus IPs (ej. webServer-01 : Servidor Web). Notar el uso del subrayado, idéntico al diagrama de objetos.

Idea Clave: Tipos de Nodos

En UML, los Nodos se dividen en dos categorías lógicas:

1. **Device (Dispositivo):** Hardware físico con capacidad de procesamiento (ej. Servidor Rack). Se marca con el estereotipo `<<device>>`.
2. **Execution Environment (Entorno de Ejecución):** Software que hospeda y ejecuta otros programas (ej. Máquina Virtual Java JVM, Contenedor Docker). Se enclava dentro de un Device.



8.6. Estructuras Compuestas (Composite Structures)

Como se aborda en las diapositivas finales, una **Estructura Compuesta** describe la organización interna y topológica de un clasificador complejo (una clase o componente), mostrando cómo sus partes colaboran para formar un comportamiento global.

A diferencia del diagrama de clases clásico (que es plano y se enfoca en las interfaces externas), el *Composite Structure Diagram* permite visualizar las "tripas" del objeto a través de:

- **Parts (Partes):** Elementos subordinados que solo existen dentro de la clase contenedora (relación de Composición estricta).
- **Ports (Puertos):** Puntos de interacción estandarizados entre el clasificador y su entorno exterior. Un puerto puede exponer explícitamente **Interfaces Ofrecidas y Requeridas** hacia el exterior.
- **Connectors (Conectores):** Líneas que trazan explícitamente cómo se comunican las partes. Existen dos tipos cruciales:
 - *Assembly Connectors (De Ensamblaje):* Unen dos partes internas entre sí.
 - *Delegation Connectors (De Delegación):* Unen un puerto externo con una parte interna, indicando qué elemento interno procesa realmente el mensaje recibido en el puerto.

Capítulo 9

Estimación de Costes: Use Case Points

9.1. El Problema Analítico de la Estimación

En la ingeniería de software, a diferencia de la ingeniería civil o la manufactura, el coste de los materiales (hardware, licencias) es despreciable frente al coste laboral. Por lo tanto, estimar el coste de un proyecto es matemáticamente equivalente a **estimar el esfuerzo humano** necesario para desarrollarlo.

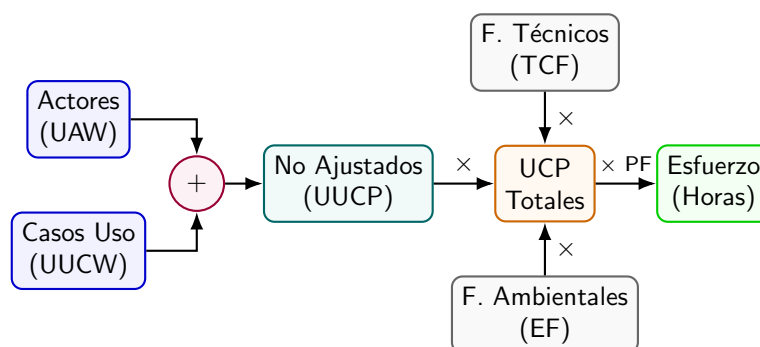
Cuando un ingeniero se enfrenta a un nuevo proyecto, debe resolver un sistema de tres incógnitas altamente acopladas:

1. **Esfuerzo (Effort):** ¿Cuántas horas-hombre se necesitan?
2. **Calendario (Schedule):** ¿Cuánto tiempo cronológico durará el proyecto?
3. **Coste (Cost):** ¿Cuál es el presupuesto financiero requerido?

Para resolver estas incógnitas de forma rigurosa y empírica en las fases tempranas del ciclo de vida, Gustav Karner propuso en 1993 una métrica basada en el modelo funcional: los **Use Case Points (UCP)**.

9.2. El Algoritmo Use Case Points (UCP)

El método UCP es una función estocástica que mapea la complejidad del sistema (descrita en los diagramas de Casos de Uso) a un valor escalar que representa el esfuerzo de desarrollo. El algoritmo consta de tres fases fundamentales.



9.2.1. Fase 1: Puntos No Ajustados (UUCP)

El tamaño intrínseco del sistema se calcula sumando la complejidad de sus actores y sus casos de uso.

$$UUCP = UAW + UUCW \quad (9.1)$$

Cálculo del UAW (Unadjusted Actor Weight)

Cada actor del diagrama se clasifica según su complejidad y se le asigna un peso discreto W_i :

- **Simple** ($W = 1$): Otro sistema informático que interactúa mediante una API programática pura.
- **Medio** ($W = 2$): Otro sistema informático que interactúa a través de un protocolo de red estándar (ej. TCP/IP, FTP) o un humano que usa una interfaz de texto plana (CLI).
- **Complejo** ($W = 3$): Un usuario humano que interactúa a través de una Interfaz Gráfica de Usuario (GUI) o web.

$$UAW = \sum(\text{Cantidad} \times \text{Peso}).$$

Cálculo del UUCW (Unadjusted Use Case Weight)

Cada Caso de Uso se clasifica según el número de **transacciones lógicas** que contiene (pasos en el escenario principal más sus escenarios alternativos/excepciones).

Definición 9.1 (Transacción Lógica). Una transacción no es un simple “clic” del usuario. Es un evento de estímulo-respuesta entre el actor y el sistema que es **atómico** (se ejecuta por completo o no se ejecuta en absoluto, dejando el sistema en un estado consistente).

- **Simple** ($W = 5$): ≤ 3 transacciones lógicas.
- **Medio** ($W = 10$): Entre 4 y 7 transacciones lógicas.
- **Complejo** ($W = 15$): > 7 transacciones lógicas.

$$UUCW = \sum(\text{Cantidad} \times \text{Peso}).$$

9.2.2. Fase 2: Factores de Ajuste Computados (TCF y EF)

La complejidad del software no depende solo de la funcionalidad estricta, sino de *cómo* debe construirse (arquitectura) y *quién* lo construye (el equipo).

Factor de Complejidad Técnica (TCF)

Existen 13 factores técnicos $T_1, \dots, T_{13}$ (ej. sistema distribuido, rendimiento crítico, concurrencia, facilidad de instalación). A cada factor se le asigna un valor de influencia $V_i \in [0, 5]$ y tiene un peso constante W_i predefinido matemáticamente por Karner.

$$TCF = 0.6 + 0.01 \cdot \sum_{i=1}^{13} (W_i \cdot V_i) \quad (9.2)$$

Factor Ambiental (EF)

Existen 8 factores ambientales $F_1, \dots, F_8$ que evalúan la calidad y aptitud del equipo de desarrollo (ej. experiencia en UML, motivación, dificultad del lenguaje de programación). Se valoran $V_i \in [0, 5]$ con pesos constantes W_i . La ecuación tiene pendiente negativa: un equipo muy experimentado reduce el tamaño aparente del problema.

$$EF = 1.4 - 0.03 \cdot \sum_{i=1}^8 (W_i \cdot V_i) \quad (9.3)$$

9.2.3. Fase 3: Puntos de Caso de Uso y Factor de Productividad (PF)

Los Use Case Points totales se obtienen escalando el tamaño base por los factores de riesgo:

$$UCP = UUCP \times TCF \times EF \quad (9.4)$$

Finalmente, para traducir la métrica abstracta UCP a un coste físico (Horas-Hombre), se multiplica por el **Factor de Productividad Empírico (PF)**.

Idea Clave: La Regla de Decisión de Karner para el PF

El valor del PF (horas necesarias para programar 1 UCP) no es arbitrario. Karner estableció un algoritmo de decisión basado en contar cuántos Factores Ambientales (EF) suponen un "riesgo":

- Si el recuento de factores de riesgo es $\leq 2 \implies \mathbf{PF = 20 \text{ horas/UCP}}$.
- Si el recuento de factores de riesgo es 3 ó 4 $\implies \mathbf{PF = 28 \text{ horas/UCP}}$ (el equipo es menos eficiente).
- Si el recuento de factores de riesgo es $\geq 5 \implies$ El riesgo es inaceptable y el proyecto debería reestructurarse.

$$\text{Esfuerzo Estimado (Horas)} = UCP \times PF \quad (9.5)$$

9.3. Limitaciones: La Paradoja de la Granularidad

Aunque los UCP proporcionan una métrica trazable, la literatura académica señala una vulnerabilidad crítica: **El Síndrome del Nivel de Detalle (Granularidad)**.

El número de casos de uso (y sus transacciones) depende drásticamente del estilo de redacción del analista:

- Si el analista redacta casos de uso muy **abstractos** (ej. "Gestionar Sistema"), el algoritmo contará pocas transacciones y **subestimar**á gravemente el esfuerzo.
- Si el analista redacta casos de uso muy **fragmentados** a nivel funcional (ej. un caso de uso para Crear, otro para Leer, otro para Actualizar, otro para Borrar - enfoque CRUD), el algoritmo **sobreestimar**á el coste del proyecto.

Por ello, los UCP requieren que la organización posea un estándar estricto y uniforme para la redacción de sus Casos de Uso.

9.4. Ejercicio Resuelto de Estimación (Tipo Examen)

Ejemplo 9.2 (Cálculo Completo de Costes en Horas). **Enunciado:** Un equipo de software DGIIM analiza un proyecto con las siguientes características:

- **Actores:** 2 usuarios web mediante Interfaz Gráfica (GUI) y 1 sistema de pago que expone una API REST.
- **Casos de Uso:** 3 Casos de Uso con 2 transacciones lógicas cada uno, 2 Casos de Uso con 5 transacciones, y 1 Caso de Uso con 9 transacciones.
- **Ajustes:** Tras evaluar las matrices, el equipo determina que $\sum(W_i \cdot V_i)$ para la parte Técnica es 40, y $\sum(W_i \cdot V_i)$ para la parte Ambiental es 10.
- **Productividad:** El análisis de riesgos ambientales arrojó solo 1 factor de riesgo, por lo que se asume un $PF = 20$.

Problema: Calcule el esfuerzo total estimado en horas de trabajo.

Resolución Matemática Paso a Paso:

Paso 1: Cálculo del UAW

- Actores GUI (Complejos): $2 \times 3 = 6$
- Actor API (Simple): $1 \times 1 = 1$
- **UAW Total:** $6 + 1 = 7$

Paso 2: Cálculo del UUCW

- Casos Simples (≤ 3 transacciones): $3 \text{ CU} \times 5 = 15$
- Casos Medios ($4 - 7$ transacciones): $2 \text{ CU} \times 10 = 20$
- Casos Complejos (> 7 transacciones): $1 \text{ CU} \times 15 = 15$
- **UUCW Total:** $15 + 20 + 15 = 50$

Paso 3: Cálculo del Puntos No Ajustados (UUCP)

$$UUCP = UAW + UUCW$$

$$UUCP = 7 + 50 = 57$$

Paso 4: Aplicación de Fórmulas TCF y EF

$$TCF = 0.6 + (0.01 \times 40) = 0.6 + 0.40 = 1.00$$

$$EF = 1.4 - (0.03 \times 10) = 1.4 - 0.30 = 1.10$$

Paso 5: Cálculo Final de Puntos Ajustados (UCP)

$$UCP = UUCP \times TCF \times EF$$

$$UCP = 57 \times 1.00 \times 1.10 = 62.7 \text{ UCP}$$

Paso 6: Conversión a Esfuerzo (Horas)

$$\text{Esfuerzo Total} = UCP \times PF$$

$$\text{Esfuerzo Total} = 62.7 \times 20 = \mathbf{1254 \text{ Horas-Hombre}}$$

Conclusión: El proyecto requerirá una estimación base de 1254 horas de trabajo neto.

Capítulo 10

Modelado Dinámico y de Comportamiento

10.1. Introducción a las Vistas Dinámicas

Los diagramas estructurales (como el Diagrama de Clases) modelan la arquitectura invariante de un sistema. Sin embargo, el software no es una entidad inerte; se ejecuta, reacciona a estímulos, cambia de estado y las partes interactúan entre sí.

Para modelar matemáticamente este comportamiento dinámico en función del tiempo (t), UML proporciona los **Diagramas de Comportamiento** (*Behavioral Diagrams*). La profesora destaca tres grandes familias formales:

1. **Diagramas de Interacción:** Modelan cómo se comunican las instancias.
2. **Diagramas de Actividad:** Modelan el flujo lógico de control y datos paso a paso.
3. **Diagramas de Máquina de Estados:** Modelan la evolución del estado interno de un único objeto en respuesta a eventos discretos.

10.2. Diagramas de Interacción

Los Diagramas de Interacción destacan por documentar cómo colaboran los diferentes objetos o actores para llevar a cabo la realización de un *Caso de Uso*. UML define oficialmente cuatro tipos de diagramas de interacción:

- **Timing Diagrams (Diagramas de Tiempos):** Se centran en restricciones temporales matemáticas muy estrictas (comunes en sistemas integrados o *hardware real-time*).
- **Interaction Overview Diagrams:** Una vista macroscópica híbrida que enruta diagramas de secuencia utilizando nodos de diagrama de actividad.
- **Diagrama de Secuencia y Diagrama de Comunicación:** Son los dos más relevantes y comúnmente aplicados en la ingeniería de software clásica.

10.2.1. El Diagrama de Secuencia (*Sequence Diagram*)

Es el diagrama dinámico más utilizado en la industria. Representa el intercambio de mensajes a lo largo del tiempo.

Características Topológicas:

- **Eje Vertical (Y):** Representa el flujo del **tiempo**, avanzando hacia abajo.

- **Eje Horizontal (X):** Representa los diferentes **objetos** o participantes.
- **Línea de Vida (Lifeline):** Línea discontinua vertical bajo un objeto que indica el tiempo que dicho objeto existe en memoria.
- **Foco de Control (Activation Box):** Rectángulo sobre la línea de vida que indica cuándo el objeto está ejecutando activamente una operación en la CPU.

Idea Clave: Ciclo de Vida: Creación y Destrucción

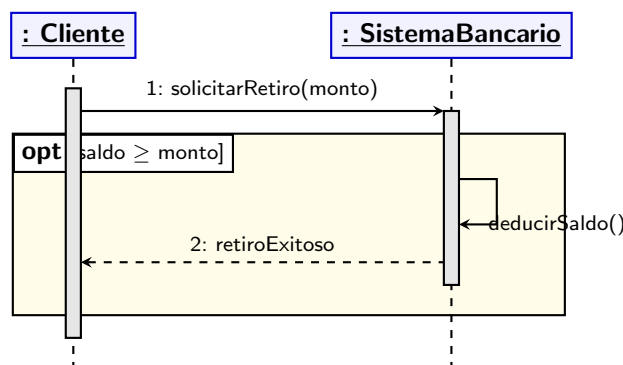
En un diagrama de secuencia, no todos los objetos existen desde el inicio.

- **Creación:** Cuando un objeto instancia a otro, la flecha del mensaje (generalmente etiquetada con <<create>>) apunta directamente al *rectángulo de cabecera* del nuevo objeto, no a su línea de vida.
- **Destrucción:** Cuando un objeto es liberado de la memoria, su línea de vida termina abruptamente con una gran **X**.

Fragmentos Combinados (Lógica Algorítmica)

En UML 2.x se introdujeron los **Fragmentos Combinados (Combined Fragments)** para permitir el modelado de lógica estructurada compleja (condicionales y bucles) dentro del diagrama de secuencia. Se dibujan como marcos rectangulares que engloban un conjunto de mensajes. Los más evaluados son:

- **opt (Opcional):** Un bloque *If* simple. El contenido solo se ejecuta si la *guarda* booleana es verdadera.
- **alt (Alternativas):** Un bloque *If-Else* o *Switch*. El marco se divide en compartimentos mutuamente excluyentes; solo se ejecuta el que cumpla su guarda.
- **loop (Bucle):** El contenido se repite iterativamente mientras la guarda sea cierta.
- **break:** Rompe y sale de un bucle inmediatamente, análogo a la instrucción en Java/C++.
- **ref:** Permite referenciar o invocar a otro diagrama de secuencia externo para no sobrecargar el actual.



Idea Clave: Tipos de Flechas de Mensaje

La punta de la flecha en UML tiene una semántica estricta:

- **Síncrono (Punta rellena ►)**: El remitente bloquea su hilo de ejecución y *espera* la respuesta del receptor antes de continuar.
- **Asíncrono (Punta abierta →)**: El remitente envía el mensaje y continúa inmediatamente su ejecución sin esperar.
- **Retorno (Línea discontinua ←--)**: Devuelve el control y/o los datos al remitente.

10.2.2. El Diagrama de Comunicación (*Communication Diagram*)

Conocido en UML 1.x como *Diagrama de Colaboración*. A diferencia del de Secuencia, ignora el eje vertical del tiempo y se centra en mostrar la red espacial de los objetos (su topología). El orden del tiempo no se infiere por la posición gráfica, sino leyendo la **numeración secuencial** de los mensajes (ej. 1.1, 1.2, 2.0).

10.3. Diagramas de Actividad

Los **Diagramas de Actividad** modelan flujos de trabajo (*workflows*) corporativos o algoritmos computacionales complejos paso a paso. Son la evolución orientada a objetos de los clásicos diagramas de flujo.

Definición 10.1 (Semántica de *Tokens* - Redes de Petri). La profesora especifica que la lógica subyacente de un diagrama de actividad se basa en el enrutamiento de *tokens* (fichas lógicas). En un **Nodo de Acción**, los *tokens* entrantes habilitan la ejecución matemática de la acción. Cuando la acción finaliza, la caja emite nuevos *tokens* por sus enlaces de salida para ceder el control al siguiente nodo de la cadena.

10.3.1. Nodos Topológicos de Control y Descomposición

Para bifurcar o unir los *tokens*, se usan nodos formales:

- **Initial Node (●)**: Inicio del flujo (crea el primer token).
- **Final Node (⊙)**: Finaliza todo el proceso destruyendo todos los tokens.
- **Decision / Merge (◇)**: Rombo. **Decision** bifurca el flujo evaluando *guardas* lógicas. **Merge** une varios caminos mutuamente excluyentes de nuevo en uno solo sin sincronizar.
- **Fork / Join (Barra negra horizontal)**: Modelan concurrencia matemática (*multi-threading*). **Fork** clona un *token* en múltiples paralelos. **Join** sincroniza caminos paralelos.

Observación 10.2 (Descomposición de Actividades). Para evitar diagramas inmanejables, UML permite la **Scomposizione** (Descomposición). Una acción puede invocar otro diagrama de actividad secundario completo. Esto se modela dibujando un pequeño símbolo de “rastrillo” (*rake symbol*) en la esquina inferior derecha del nodo de acción.

10.3.2. Flujo de Objetos (*Object Flow*) y Estado

Además de enrutar el control de ejecución, las actividades procesan datos. UML permite modelar explícitamente esto mediante **Nodos de Objeto** (rectángulos simples) unidos por flujos de objetos. Un detalle teórico crucial que pide la profesora es que los nodos de objeto pueden indicar el **estado del objeto en ese preciso instante** escribiéndolo entre corchetes.

Ejemplo de flujo: Pedido [Pendiente] → (Nodo Acción: Facturar) → Pedido [Pagado].

10.3.3. Señales y Eventos (Signals)

A diferencia de las Máquinas de Estado donde los eventos son centrales, en los diagramas de actividad el flujo se rige por su propia finalización. Sin embargo, a veces el flujo es alterado por el mundo exterior. Para ello, existen 3 figuras clave de **Señales**:

1. **Send Signal (Polígono convexo $\rangle$)**: El sistema emite un evento asíncrono al exterior y continúa inmediatamente.
2. **Accept Signal (Polígono cóncavo $\langle$)**: El sistema detiene el token y queda en letargo hasta recibir un evento externo esperado.
3. **Time Event / Timer (Reloj de arena $\bowtie$)**: Congela la ejecución del token hasta que se cumpla una condición temporal estricta (ej. “Esperar 2 horas”, “El 1er día del mes”).

10.3.4. Participaciones: Swimlanes

Los Diagramas de Actividad permiten subdividir el lienzo en carriles (columnas o filas) llamados **Swimlanes**. Cada carril representa a un actor, clase o departamento específico. Es una herramienta de diseño vital porque define explícitamente **quién tiene la responsabilidad** de ejecutar cada acción matemática del flujo.

10.4. Diagrama de Máquina de Estados (State Machine)

A diferencia de los diagramas de actividad que modelan el comportamiento global de un proceso, la **Máquina de Estados** (*Statechart Diagram*) describe el ciclo de vida determinista de **un único objeto** a lo largo del tiempo en respuesta a estímulos matemáticos.

Se utiliza principalmente para modelar objetos complejos guiados por eventos (*Event-Driven*), como el controlador de una lavadora, un microondas o la conexión de un socket de red TCP.

- **Estado**: Una condición matemática en la que el objeto satisface algún predicado (ej. **Encendido**, **Bloqueado**). Puede ejecutar *acciones internas* al entrar (**entry/**), al salir (**exit/**) o de forma continua (**do/**).
- **Transición**: Un cambio lógico entre un estado de origen y un estado destino.

Idea Clave: Sintaxis Estricta de una Transición

La flecha de transición se etiqueta rigurosamente bajo la siguiente función lógica:

Evento [Condición de Guarda] / Acción Ejecutada

- **Evento:** El estímulo o *trigger* que despierta la transición (ej. pulsar un botón).
- **Guarda:** Una expresión booleana `true/false`. Si el evento ocurre, pero la guarda es `false`, la transición **NO** se dispara y el evento se ignora.
- **Acción:** Procedimiento de software inmediato y atómico que se ejecuta durante el viaje topológico de un estado a otro.

10.4.1. Conceptos Avanzados: Estados Compuestos y Regiones Ortogonales

Para gobernar la explosión combinatoria de estados en sistemas grandes, UML soporta abstracciones complejas:

- **Estado Compuesto:** Un estado que alberga internamente otra máquina de estados completa (sub-estados). Se utiliza para encapsular comportamientos anidados.
- **Regiones Ortogonales:** Cuando un Estado Compuesto posee múltiples flujos de estados ejecutándose concurrentemente en paralelo, la caja del estado se divide mediante líneas discontinuas, formando regiones ortogonales.

Capítulo 11

Calidad del Software: Estándar ISO/IEC 25010

11.1. Introducción a la Calidad en la Ingeniería

En las fases tempranas de la computación, la "calidad" del software se medía heurística y subjetivamente (ausencia de cuelgues o rapidez aparente). Sin embargo, para que la Ingeniería del Software alcance la madurez de disciplinas como la aeronáutica o la civil, la calidad debe ser **sistemática, cuantificable y estandarizada**.

Definición 11.1 (Calidad del Software - IEEE). El IEEE define formalmente la Calidad del Software como: «*El grado en el que un sistema, componente o proceso cumple con los requisitos especificados y satisface las necesidades o expectativas declaradas e implícitas del cliente o usuario final.*»

11.1.1. El Antecesor Histórico: El Modelo de McCall (1977)

Antes de la estandarización ISO, el marco de referencia académico era el **Modelo de McCall**. Este modelo sentó las bases de la calidad moderna al dividir axiomáticamente los atributos del software en tres dimensiones temporales (perspectivas):

1. **Operación del Producto (Presente):** Lo que experimenta el usuario al usarlo (Corrección, Fiabilidad, Eficiencia, Integridad, Usabilidad).
2. **Revisión del Producto (Futuro Cercano):** El esfuerzo requerido para encontrar y corregir errores (Mantenibilidad, Flexibilidad, Testabilidad).
3. **Transición del Producto (Futuro Lejano):** El esfuerzo requerido para adaptar el software a nuevos entornos de hardware o integrarlo con otros sistemas (Portabilidad, Reusabilidad, Interoperabilidad).

11.2. La Familia SQuaRE (ISO/IEC 25000)

Para unificar los diversos modelos (como McCall o el antiguo ISO 9126), la Organización Internacional de Normalización (ISO) y la Comisión Electrotécnica Internacional (IEC) desarrollaron un marco matemático y semántico unificado.

Definición 11.2 (SQuaRE). La familia de estándares **ISO/IEC 25000** recibe el acrónimo **SQuaRE** (*System and Software Quality Requirements and Evaluation*). Su objetivo es proporcionar un marco lógico para especificar y evaluar exhaustivamente los requisitos de calidad de un producto de software.

Para cubrir todas las facetas de la ingeniería, SQuaRE se divide en **cinco divisiones matriciales**:

- **ISO 2500n:** División de Gestión de la Calidad.
- **ISO 2501n:** División de **Modelos de Calidad** (donde residen las características).
- **ISO 2502n:** División de Medición de la Calidad (métricas matemáticas exactas).
- **ISO 2503n:** División de Requisitos de Calidad.
- **ISO 2504n:** División de Evaluación de la Calidad.

Idea Clave: Ingeniería de Diseño y Calidad

La profesora plantea una pregunta fundacional: «¿Cómo se obtienen las características de calidad?». La calidad no aparece por arte de magia al final del proyecto; **se inyecta estructuralmente durante las fases de diseño y arquitectura.**

- Los principios de modularidad y bajo acoplamiento garantizan la **Mantenibilidad**.
- Elegir un *Estilo de Arquitectura en Capas* maximiza la **Modificabilidad**.
- Elegir un *Estilo de Microservicios* garantiza la **Reusabilidad y Escalabilidad**.
- Implementar un *Load Balancer* mejora la **Eficiencia de Desempeño**.

11.3. Evolución Normativa: De ISO 9126 a ISO 25010

El estándar actual en vigor en la industria es el **ISO/IEC 25010** (2011), el cual reemplazó y extendió al antiguo estándar ISO 9126. Las diferencias y actualizaciones fundamentales exigidas son:

1. **Revolución en la Calidad en Uso:** Las características que definen el impacto del software en el mundo real han sido completamente rediseñadas y ampliadas.
2. **Promoción de Categorías Críticas:** La **Seguridad** (*Security*) y la **Compatibilidad** (*Compatibility*) dejaron de ser sub-características secundarias para ser "promovidas" a características principales de primer nivel.
3. **Eliminación de la Conformidad:** Los requisitos de conformidad (*Compliance*) desaparecieron como categoría independiente, asumiéndose que el cumplimiento de estándares permea todo el sistema.

11.4. La Dicotomía del Modelo ISO 25010

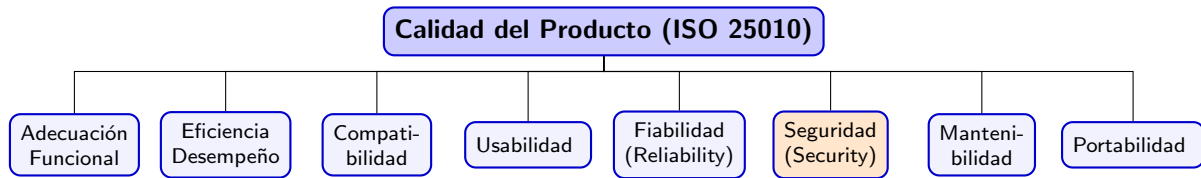
El estándar ISO 25010 divide el universo de la calidad en dos modelos ortogonales pero complementarios:

- **Modelo de Calidad del Producto:** Evalúa las propiedades intrínsecas y estáticas/dinámicas del sistema computacional en sí mismo (visión del ingeniero).
- **Modelo de Calidad en Uso:** Evalúa el grado en el que el producto permite a usuarios específicos alcanzar sus objetivos con eficacia, seguridad y satisfacción en un contexto de uso determinado (visión del usuario final).

Observación 11.3 (La Importancia de los Datos: ISO 25012). El estándar advierte que un software algorítmicamente perfecto puede fracasar estrepitosamente si procesa datos erróneos. Por ello, SQuaRE incluye el estándar complementario **ISO/IEC 25012 (Modelo de Calidad de Datos)**, que evalúa de forma independiente si la información almacenada en el sistema es *Precisa, Completa, Consistente y Actualizada*.

11.4.1. 1. Modelo de Calidad del Producto (*Product Quality*)

Este modelo categoriza las propiedades computacionales del software en **8 características principales**.



Definiciones Claves y sus Sub-características:

- **Adecuación Funcional (*Functional Suitability*)**: Grado en que las funciones satisfacen las necesidades. Sub-características: *Compleitud* (¿está todo?), *Corrección* (¿es exacto?) y *Pertinencia* (¿es apropiado?).
- **Eficiencia de Desempeño (*Performance Efficiency*)**: Rendimiento en relación a los recursos. Sub-características: *Comportamiento Temporal* (tiempos de respuesta), *Utilización de Recursos* y **Capacidad** (*Capacity*: límites máximos soportados).
- **Compatibilidad**: Capacidad de intercambiar información. Sub-características: *Interoperabilidad* (comunicación con otros) y *Coexistencia* (compartir hardware sin interferir).
- **Usabilidad**: Facilidad de uso. Sub-características críticas: *Capacidad de Aprendizaje* (Learnability), *Protección contra errores del usuario*, **Accesibilidad** (uso por personas con capacidades diferentes) y **Estética de la Interfaz**.
- **Fiabilidad (*Reliability*)**: Mantener el rendimiento en el tiempo. Sub-características críticas: *Madurez*, *Disponibilidad*, **Tolerancia a Fallos** (seguir operando pese a fallos hardware/software) y **Recuperabilidad** (restaurar datos tras un fallo).
- **Seguridad (*Security*)**: Protección de la información. Sub-características: *Confidencialidad*, *Integridad*, *No Repudio*, *Responsabilidad (Accountability)* y *Autenticidad*.
- **Mantenibilidad**: Eficiencia para modificar el código. Sub-características: *Modularidad*, *Reusabilidad*, *Analizabilidad*, *Modificabilidad*, *Testabilidad*.
- **Portabilidad**: Facilidad de transferencia. Sub-características críticas: *Adaptabilidad*, **Instalabilidad** (eficiencia al instalar/desinstalar) y **Reemplazabilidad** (sustituir un software por otro con igual propósito).

11.4.2. 2. Modelo de Calidad en Uso (*Quality in Use*)

Responde a la pregunta: «¿Cuál es su impacto final en el mundo humano?» Está compuesto por **5 características principales**:

1. **Efectividad (*Effectiveness*)**: Precisión y completitud con la que los usuarios alcanzan sus objetivos reales.
2. **Eficiencia (*Efficiency*)**: Recursos gastados por el humano (tiempo, esfuerzo mental) en relación a la precisión y completitud lograda.

3. **Satisfacción (*Satisfaction*):** Grado en el que las necesidades del usuario son satisfechas. Se subdivide en: *Trust* (Confianza ciega en el sistema), *Pleasure* (Placer personal) y *Comfort* (Comodidad y ergonomía).
4. **Libertad de Riesgo (*Freedom from Risk*):** Capacidad del sistema para mitigar el riesgo potencial en tres áreas:
 - Mitigación de daños al *estado económico*.
 - Mitigación de daños a la *salud humana o la vida* (Safety).
 - Mitigación de daños al *medio ambiente*.
5. **Cobertura de Contexto (*Context Coverage*):** Grado en el que el producto puede usarse con calidad en los contextos originales y en contextos imprevistos (Flexibilidad).

11.5. El Ciclo de Vida de la Evaluación de la Calidad

Las métricas de SQuaRE no se evalúan todas al mismo tiempo, sino que siguen una progresión natural causal:

1. **Calidad Interna:** Se evalúa sobre representaciones **estáticas** del software (documentos de requisitos, modelos UML, código fuente). Usa métricas de complejidad ciclomática, acoplamiento, etc. No requiere ejecutar el programa.
2. **Calidad Externa:** Se evalúa sobre el software **dinámico** en ejecución en un entorno de pruebas (simulado o de *testing*). Se evalúa el rendimiento, uso de RAM, tiempos de respuesta y estrés.
3. **Calidad en Uso:** Se evalúa únicamente cuando el software final es operado por el **usuario real en su contexto operativo real**.

La premisa matemática de la ingeniería es que la Calidad Interna es un *prerrequisito necesario (pero no suficiente)* para la Calidad Externa, y la Calidad Externa es el puente ineludible hacia la Calidad en Uso.

Ejemplo 11.4 (Evaluación Práctica de Calidad Interna). La profesora presenta un fragmento de código Python real para ilustrar empíricamente cómo una mala técnica de programación impacta directamente en las métricas de la norma ISO 25010:

```
def calculate_price(user_type, product_price):
    if user_type == "student":
        final_price = product_price - (product_price * 0.1)
    elif user_type == "teacher":
        final_price = product_price - (product_price * 0.2)
    elif user_type == "employee":
        final_price = product_price - (product_price * 0.15)
    elif user_type == "vip":
        final_price = product_price - (product_price * 0.25)
    return final_price
```

Análisis de Calidad Estructural:

- **Mantenibilidad (Modificabilidad y Analizabilidad):** Este código sufre de una pésima calidad interna. Posee una alta *Complejidad Ciclomática* (demasiados saltos lógicos if-elif). Si en el futuro la empresa exige añadir un nuevo perfil (ej. "jubilado"), el ingeniero está obligado a modificar el núcleo matemático de la función.
- **Violación de Principios de Diseño:** Vulnera el Principio de Abierto/Cerrado (*Open/Closed Principle*).
- **Testabilidad:** Probar todas las ramas requiere múltiples casos de test que crecerán indefinidamente con cada nuevo rol.

Este ejemplo demuestra axiomáticamente que **un diseño deficiente degrada la Calidad Interna, lo cual impide lograr los estándares de la Calidad del Producto (ISO 25010).**

Capítulo 12

Diseño y Arquitectura de Software

12.1. El Proceso de Diseño (*Progettazione*)

Si el análisis de requisitos responde al "qué" debe hacer el sistema, el diseño (*progettazione*) responde al "cómo" lo hará computacionalmente. Es el puente ineludible entre los modelos conceptuales abstractos y la implementación en código real.

Definición 12.1 (Diseño de Software). Proceso creativo e iterativo en el cual los requisitos del sistema se traducen en una representación de la arquitectura, componentes, interfaces y datos, estableciendo la estructura fundamental del software antes de su codificación. Su objetivo es reducir los costes de desarrollo y asegurar la calidad en el tiempo.

El diseño se divide clásicamente en dos fases:

1. **Diseño Arquitectónico (Macroscópico):** Define la estructura global, identificando los grandes subsistemas, paquetes o componentes y sus relaciones.
2. **Diseño Detallado (Microscópico):** Especifica la estructura interna de cada componente, las clases concretas, sus métodos, algoritmos y estructuras de datos.

12.1.1. Principios Fundamentales del Diseño

Para que un diseño posea alta calidad interna (Mantenibilidad, Fiabilidad), debe regirse por axiomas ingenieriles inquebrantables:

Idea Clave: Los 4 Pilares del Diseño de Módulos

- **Modularidad:** Dividir un sistema monolítico inmanejable en partes más pequeñas y manejables (módulos o componentes) que pueden diseñarse y probarse de forma semi-independiente. En Java, los módulos son clases o paquetes.
- **Information Hiding (Ocultamiento de Información):** Principio dictado por Parnas. Un módulo debe ocultar sus detalles de implementación interna (estructuras de datos, algoritmos) y exponer únicamente una interfaz pública estable y bien definida.
- **Alta Cohesión (*High Cohesion*):** Medida intra-módulo. Un módulo es altamente cohesivo si todos sus elementos internos colaboran estrictamente para cumplir una **única responsabilidad** bien definida. Evita las "Clases Dios" (*God Classes*).
- **Bajo Acoplamiento (*Low Coupling*):** Medida inter-módulo. Es el grado de interdependencia matemática entre módulos. Se busca minimizarlo: un cambio en el Módulo A no debería forzar una reescritura en el Módulo B.

Observación 12.2 (Diseño para la Prevención de la Obsolescencia). La profesora advierte rigurosamente que un buen proyecto **no debe ser apto solo para el presente, sino resistente a los cambios futuros**. Para ello dicta heurísticas estrictas:

- **Evitar el Vendor Lock-in:** No depender de hardware o software propietario de empresas que podrían dejar de dar soporte.
- **Evitar librerías excesivamente específicas:** Preferir siempre soluciones estándar y lenguajes soportados por múltiples proveedores.
- **Rechazar características no documentadas:** Jamás utilizar funciones oscuras o experimentales (*beta*) de los entornos de programación.

Observación 12.3 (La Regla de Oro del Testing). La profesora enfatiza el concepto de **Diseñar para la Testabilidad**. Un buen diseño aísla la lógica de negocio de la interfaz gráfica (GUI) o de la base de datos, permitiendo inyectar *tests automáticos* (ej. con JUnit) que verifiquen el núcleo matemático del software de forma independiente.

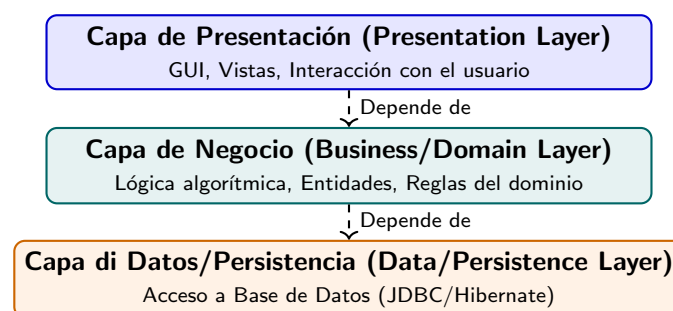
12.2. Patrones Arquitectónicos (*Architectural Patterns*)

Definición 12.4 (El Origen de los Patrones - Christopher Alexander). El concepto de patrón no nació en la informática, sino en la arquitectura de edificios. Según C. Alexander: «*Un patrón describe un problema que ocurre una y otra vez en nuestro entorno, y luego describe el núcleo de la solución a ese problema, de tal manera que puedes usar esta solución un millón de veces sin hacerla de la misma forma dos veces.*»

Un **Patrón Arquitectónico** es la aplicación de esta filosofía a nivel macroestructural del software. Dicta cómo se organizan los subsistemas a gran escala.

12.2.1. El Patrón de Arquitectura en Capas (*Layered Architecture*)

Es el patrón fundacional más utilizado. Organiza el sistema en estratos lógicos superpuestos. La regla matemática estricta es: **una capa N solo puede depender y comunicarse con la capa inferior $N - 1$ (o inferiores)**.



Ventaja Crítica: *Separation of Concerns* (Separación de responsabilidades). Permite cambiar la base de datos (capa inferior) sin tocar la interfaz gráfica (capa superior).

12.2.2. El Enfoque BCED (Boundary-Control-Entity-Data)

El **BCED** es una especialización pragmática del patrón de capas y del patrón MVC clásico, fuertemente promovido por la profesora para organizar el código Java. Asigna responsabilidades semánticas a las clases basándose en el análisis de *Casos de Uso*.

Las clases se agrupan formalmente en cuatro *packages*:

1. **boundary**: Clases que gestionan el I/O con los actores externos (ej. ventanas *Swing* en Java). No contienen lógica de negocio.
2. **control**: Clases que orquestan el flujo lógico de un Caso de Uso específico. Reciben eventos de la Boundary, toman decisiones y coordinan a las Entities.
3. **entity**: Clases que representan los conceptos ontológicos del mundo real (ej. **Factura**, **Usuario**). Contienen los datos (estado) e las operaciones matemáticas puras del negocio.
4. **data (o persistence)**: Clases abstractas o concretas (como los DAOs - *Data Access Objects*) que encapsulan las queries SQL o llamadas a *Hibernate*.

12.3. Patrones de Asignación de Responsabilidades: GRASP

Antes de aplicar los complejos patrones GoF, el ingeniero debe dominar los principios básicos de diseño orientado a objetos. Estos principios fueron empaquetados por Craig Larman como patrones **GRASP** (*General Responsibility Assignment Software Patterns*).

La profesora exige dominar estos tres patrones fundacionales para la arquitectura BCED:

- **Information Expert (Experto en Información)**: Responde a la pregunta: «¿Qué clase posee la información necesaria para realizar una operación?». La responsabilidad debe asignarse a la clase que posea los datos necesarios para completarla. (Por ejemplo, el cálculo del total de un pedido debe ir en la clase **Pedido**, no en la interfaz gráfica).
- **Creator (Creador)**: Responde a la pregunta: «¿Quién debe crear una nueva instancia de la clase A?». Se le asigna la responsabilidad a una clase B si: B agrega o compone a A, B contiene los datos de inicialización de A, o B usa estrechamente a A.
- **Controller (Controlador)**: Responde a la pregunta: «¿Cuál es el primer objeto más allá de la GUI que recibe y coordina una operación de sistema?». **Solución**: Un evento del usuario es interceptado por la GUI (**boundary**), pero esta no debe procesarlo. Debe delegar la llamada a un objeto **Controller** que orqueste la respuesta.

12.4. Patrones de Diseño (*Design Patterns* - *GoF*)

Formalizados por la *Gang of Four* - *GoF* en 1994, resuelven problemas microestructurales a nivel de clases y objetos.

Idea Clave: La Matriz de Clasificación GoF

El catálogo GoF se clasifica rigurosamente mediante una matriz bidimensional:

- **Propósito (*Purpose*)**: **Creacional** (instanciación), **Estructural** (composición) y **Comportamental** (comunicación).
- **Alcance (*Scope*)**: **Clase** (las relaciones se definen estáticamente en tiempo de compilación mediante *Herencia*) y **Objeto** (las relaciones se definen dinámicamente en tiempo de ejecución mediante *Composición*).

12.4.1. Patrones Creacionales

1. Singleton (Scope: Objeto)

Problema: Garantizar que una clase tenga **exactamente una única instancia** global y proveer un punto de acceso global a ella.

Solución en Java: Constructor privado, variable estática privada y método estático público (`getInstance()`) con *Lazy Initialization*.

2. Factory Method (Scope: Clase)

Problema: Una superclase necesita instanciar objetos derivados, pero no conoce a priori qué subclase exacta debe crear.

Solución: Definir un método abstracto para crear un objeto, permitiendo que sean las **subclases las que decidan por herencia qué clase concreta instanciar**.

12.4.2. Patrones Estructurales

3. Adapter (Scope: Clase u Objeto)

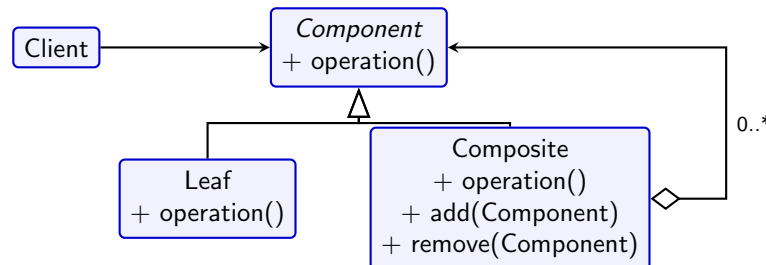
Problema: Dos clases incompatibles deben colaborar (una clase *Adaptee* no encaja con lo que el Cliente espera).

Solución: Crear una clase *Adapter* que actúe como traductor. Implementa la interfaz que el Cliente espera y "envuelve" las llamadas hacia el objeto Adaptee.

4. Composite (Scope: Objeto)

Problema: Representar jerarquías recursivas *árbol-hoja* tratando a hojas y contenedores uniformemente.

Solución: Definir una interfaz común (*Component*). Tanto hojas simples (*Leaf*) como contenedores (*Composite*) implementan esta interfaz. El *Composite* delega iterativamente las operaciones a sus hijos.



12.4.3. Patrones Comportamentales

5. Observer (Scope: Objeto)

Problema: Mantener la consistencia entre objetos acoplados. Cuando el Sujeto cambia, sus Observadores deben ser notificados automáticamente.

Solución: El Sujeto mantiene una lista de referencias a interfaces abstractas **Observer**. Cuando ocurre un cambio, invoca `update()` en cada uno.

6. State (Scope: Objeto)

Problema: El comportamiento de un objeto depende completamente de su estado interno, cambiando dinámicamente.

Solución: Encapsular cada estado en una clase distinta que implementa una interfaz de **Estado**.

Ejemplo: El motor de una puerta automática delega su comportamiento a los objetos **EstadoAbierta**, **EstadoCerrada**, etc., evitando inmensos bloques `if-else`.

12.5. Traducción Lógica: De UML a Java

La profesora establece normativas estrictas para traducir los modelos estáticos UML a código fuente Java compilable.

12.5.1. Mapeo de Interfaces (Provided vs. Required)

- **Provided Interface (Lollipop):** Se traduce en una `interface` Java pura, junto con la clase concreta que la implementa (`implements`).
- **Required Interface (Socket):** Se traduce topológicamente como una variable de instancia (atributo) del tipo de la interfaz dentro de la clase cliente, que será inyectada mediante el constructor.

12.5.2. Traducción Analítica de Asociaciones y Colecciones

- **Asociaciones 1:1 o 1:0..1** Se mapean como referencias de objeto simples (ej. `private Badge badge;`).
- **Asociaciones 1:N o N:M** Se mapean utilizando el API de Colecciones de Java, comúnmente una interfaz `Set<T>` o `List<T>`.

Idea Clave: El Peligro del Mapeo con Set en Java

Un error estructural común penalizado en exámenes ocurre al mapear asociaciones utilizando conjuntos sin duplicados (`Set<T>`). Si una clase `Corso` guarda sus estudiantes en un `Set<Studente>`, es **algorítmicamente obligatorio sobrescribir los métodos `equals()` y `hashCode()`** en la clase `Studente`. De lo contrario, Java comparará las direcciones de memoria de los objetos en lugar de sus identificadores lógicos (como la *matricola*), corrompiendo la asociación al permitir estudiantes duplicados.

12.5.3. Reglas Generales de Traducción Adicionales

La profesora añade un par de heurísticas para no errar en el traspaso de UML a código:

- **Encapsulación por defecto:** Todos los atributos en Java deben ser declarados `private` salvo justificación excepcional, exponiendo *getters* y *setters* (si la lógica lo permite). Esto alinea el código con la filosofía de *Information Hiding*.
- **Métodos de Colecciones:** Si una clase *A* posee una lista de clases *B* (`List<B>`), la clase *A* no debe exponer el *getter* de la lista completa (que podría ser modificado externamente). Debe ofrecer métodos semánticos encapsulados: `addB(B obj)` y `removeB(B obj)`.

12.6. Implementación BCED Práctica: boatyard-demo

Para llevar el diseño BCED a la realidad, la profesora exige el uso de **IntelliJ IDEA** y **Maven**. En el proyecto práctico del astillero (`it.unina.boatyard`):

El Rol del Boundary y la Validación: Una regla vital es que la interfaz gráfica (`boundary` en Java Swing) debe realizar las validaciones sintácticas básicas **antes** de invocar al `control`. En el código fuente, la vista verifica si el campo de texto está vacío e invoca un `JOptionPane.showMessageDialog(...)` para detener el flujo inmediatamente.

JDBC vs Hibernate en la capa Data:

- **JDBC:** API de bajo nivel. Requiere escribir sentencias SQL a mano y mapear cada columna de un `ResultSet` a atributos.

- **Hibernate (Framework ORM):** Marco de **Mapeo Objeto-Relacional**. Permite manipular directamente objetos Java (*Entities*); Hibernate traduce estas operaciones a consultas SQL automáticamente, resolviendo el problema de la asimetría de paradigmas.

Capítulo 13

Verificación, Validación y Testing

13.1. Fundamentos de Calidad Dinámica: V&V

A lo largo del ciclo de vida del software, es imprescindible garantizar matemáticamente y empíricamente que el sistema que se está construyendo es correcto y útil. Toda disciplina ingenieril asocia actividades de construcción con actividades de prueba. Este proceso dual se conoce formalmente como **Verificación y Validación (V&V)**.

Aunque a menudo se usan como sinónimos en el lenguaje coloquial, en la Ingeniería del Software (y según la norma de la IEEE) representan dos preguntas diametralmente opuestas, formuladas por B. Boehm:

- Definición 13.1** (Verificación vs. Validación). - **Verificación (*Verification*)**: «¿Estamos construyendo el producto correctamente?». Es el proceso de evaluar si el software cumple estrictamente con las especificaciones técnicas, la arquitectura y los modelos UML definidos en la fase de diseño.
- **Validación (*Validation*)**: «¿Estamos construyendo el producto correcto?». Es el proceso de evaluar el software al final del desarrollo para asegurar que satisface los verdaderos requisitos, necesidades y expectativas del cliente en el mundo real.

13.1.1. La Dificultad Intrínseca: Peculiaridades del Software

La profesora destaca que realizar la V&V en el software es matemáticamente más difícil que en la ingeniería clásica debido a tres factores intrínsecos:

1. **No-linealidad matemática**: En la ingeniería civil, si un ascensor soporta 1000 Kg, es obvio que soportará cualquier peso menor (es lineal). Sin embargo, en el software, si un algoritmo ordena correctamente un array de 256 elementos, **no hay garantía matemática** de que ordene bien un array de 255 o de 12 elementos. Un simple cambio de estado rompe la linealidad, requiriendo pruebas exhaustivas.
2. **Estructura Evolutiva**: El software experimenta un "deterioro" lógico (deuda técnica) con el tiempo a medida que sufre modificaciones y parches sucesivos.
3. **Requisitos Contrastantes**: Deben conciliarse atributos de calidad que chocan entre sí (por ejemplo, exigir máxima seguridad de cifrado frente a exigir máxima velocidad de procesamiento).

13.2. Niveles de Granularidad del Testing

El Testing no es una fase monolítica que se ejecuta exclusivamente al final del proyecto. Según las lecciones de la profesora, el testeo dinámico se aplica en distintos **niveles de granularidad** para acorralar los errores desde lo microscópico hasta lo macroscópico.

1. **Testing de Unidad (*Unit Testing*):** Se evalúan módulos aislados o clases individuales. Se basa en las especificaciones de las interfaces de los módulos o del diseño detallado.
2. **Testing de Integración (*Integration Testing*):** Evalúa cómo los módulos previamente testeados por unidad interactúan entre sí. Detecta errores en las interfaces y en el intercambio de datos (APIs).
3. **Testing de Sistema (*System Testing*):** Se evalúa el software completo y ensamblado como una caja negra frente a los Requisitos de Sistema (SRS) originales.
4. **Testing de Aceptación (*Acceptance Testing*):** Realizado por el cliente o los usuarios finales para validar si el sistema está listo para su despliegue comercial. Se divide formalmente en:
 - **Alpha-test:** Realizado por el cliente pero en un entorno controlado (usualmente dentro de las instalaciones del desarrollador).
 - **Beta-test:** Realizado por un grupo de usuarios finales en su entorno operativo real, fuera del control de los desarrolladores.
5. **Testing de Regresión (*Regression Testing*):** Prueba que se ejecuta repetidamente (generalmente automatizada) cada vez que se introduce un cambio o se corrige un bug en el código, para garantizar que la nueva modificación no ha "roto" funcionalidades antiguas que ya funcionaban.

13.3. Mecánica del Testing de Unidad y de Integración

Dado que los sistemas modernos son modulares, es imposible testear un componente en el vacío si este depende lógicamente de otros módulos que aún no han sido programados. Para resolver este bloqueo topológico, la ingeniería introduce los **Drivers** y los **Stubs (Módulos Ficticios)**.

Idea Clave: Módulos Ficticios: Drivers y Stubs

- **Driver (Módulo Piloto):** Es un módulo escrito exclusivamente para la prueba que **simula a los módulos superiores (llamadores)**. Instancia el módulo bajo test, le pasa los casos de prueba (datos de entrada) y recoge/imprime los resultados.
- **Stub (Módulo Ficticio):** Es un módulo que **simula a los módulos inferiores (subordinados)**. Cuando el módulo bajo test hace una llamada a un sub-módulo que aún no existe, el Stub recibe la llamada y devuelve datos "quemados" (*hardcoded*) para permitir que la ejecución continúe.

13.3.1. Estrategias de Integración Continua

Cuando pasamos del Test de Unidad al Test de Integración, el orden en el que ensamblamos el grafo de módulos es crítico. Existen dos enfoques matemáticos principales:

1. Integración Top-Down (Descendente)

El ensamblaje comienza por el módulo principal de control (*MAIN*) y desciende por la jerarquía.

- Puede hacerse en profundidad (*Depth-first*) o en anchura (*Breadth-first*).
- **Ventaja:** La lógica de control principal (el esqueleto) se valida tempranamente.
- **Desventaja Crítica:** Requiere la creación masiva de **Stubs** para simular todos los niveles inferiores, lo cual es laborioso y puede ocultar errores de precisión si los Stubs devuelven datos demasiado simplificados.

2. Integración Bottom-Up (Ascendente)

El ensamblaje comienza por los módulos de nivel más bajo (hojas del árbol que no tienen dependencias subordinadas) y agrupa los módulos en *clusters* funcionales ascendiendo hacia el *MAIN*.

- **Ventaja:** Elimina por completo la necesidad de **Stubs**, ya que los submódulos reales siempre están terminados antes que sus superiores.
- **Desventaja Crítica:** Requiere la creación de **Drivers** para orquestar la ejecución temporal de los clusters. La lógica de control central no se prueba hasta el final del proyecto. Además, la profesora alerta del **problema del "scarica barile"** (**elusión de responsabilidades**): al no probarse el sistema global hasta el final, si un error emerge en las altas capas integrativas, los desarrolladores de los módulos inferiores tenderán a culparse mutuamente.

13.4. Enfoques Metodológicos: Caja Blanca vs Caja Negra

Las técnicas matemáticas para generar los Casos de Prueba (*Test Cases*) se dividen topológicamente en dos escuelas: Testing Funcional y Testing Estructural.

Idea Clave: El Horizonte del Testing

- **Caja Blanca (Testing Estructural):** Conoce la arquitectura interna. Se aplica a pequeñas partes del código (Test de Unidad). Su objetivo es recorrer el flujo algorítmico, probando ramas lógicas (*if-else*), bucles y asegurando una alta *Cobertura de Código*.
- **Caja Negra (Testing Funcional):** Ignora por completo la estructura interna. Considera al sistema como una caja estanca opaca. La fuente de información para diseñar los tests son **exclusivamente las especificaciones de requisitos**. Inyecta *Inputs* y valida que los *Outputs* correspondan a la función matemática deseada.
Nota teórica: El término "Funcional" aquí se refiere estrictamente a la **fuentes de información** usada para derivar los casos de prueba (las especificaciones), no a qué aspecto del sistema se está testeando.

13.4.1. La Limitación Axiomática de la Caja Blanca

En los apuntes de la asignatura se formula una advertencia crítica fundamental de ingeniería: **El test estructural (Caja Blanca) jamás se focalizará en código que no existe**. Si durante la fase de desarrollo un ingeniero olvidó por completo implementar un requisito del cliente (un olvido funcional), los tests de Caja Blanca pasarán todos con éxito al 100 % porque solo evalúan el código existente. Por lo tanto, el testing estructural por sí solo es inválido, requiriendo obligatoriamente del Testing Funcional para detectar omisiones de diseño.

Taxonomía de Errores Estructurales: Cuando se aplica la Caja Blanca, el objetivo es detectar 5 categorías precisas de defectos:

1. Funciones con implementaciones incompletas o faltantes (ej. etiquetas TODO olvidadas en el código de producción).
2. Errores de interfaz (datos incoherentes pasados entre módulos).
3. Errores en las estructuras de datos o en los accesos a bases de datos externas.
4. Errores en la lógica de presentación (el código que dibuja las interfaces).
5. Errores matemáticos de inicialización y terminación de variables.

13.4.2. Testing Funcional: Partición Sistemática (Partition Testing)

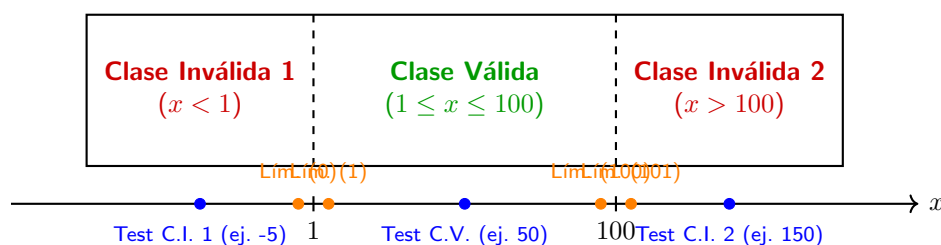
El mayor problema del Testing Funcional es que el dominio de entrada de un programa (los valores posibles que el usuario puede teclear) es, a efectos prácticos, **infinito**. Es imposible probar todos los números enteros en una función suma, o realizar lo que se conoce como **Random Testing**, ya que la posibilidad de encontrar fallos específicos sin un orden lógico sería ineficiente y computacionalmente inviable.

Para resolver esto, la ingeniería utiliza el **Systematic Partition Testing** (Partición de Clases de Equivalencia).

Definición 13.2 (Clases de Equivalencia). Una clase de equivalencia es un subconjunto matemático del dominio de entrada para el cual se asume formalmente que el comportamiento del software será idéntico. *Si un caso de prueba dentro de una clase revela un error, se asume que todos los demás valores de esa misma clase también revelarán el mismo error.*

Metodología de diseño de Test Cases:

1. El dominio se particiona en **Clases Válidas** (entradas que el sistema debe aceptar) y **Clases Inválidas** (entradas que el sistema debe rechazar lanzando excepciones).
2. Se selecciona **un único valor representativo** en el centro de cada clase de equivalencia para redactar el test.
3. *Análisis de Valores Límite (Boundary Value Analysis)*: La evidencia empírica demuestra que la inmensa mayoría de los fallos de software (ej. *Off-by-one errors* o usar $<$ en lugar de $\leq$) ocurren en los bordes matemáticos de las clases de equivalencia (los límites). Por lo tanto, se deben añadir *Test Cases* explícitos para el valor mínimo, el valor máximo, y los valores inmediatamente adyacentes al límite (fuera de la clase).



Capítulo 14

Prácticas Avanzadas: Clean Code, SOLID y Herramientas

14.1. Introducción a las Best Practices

A lo largo de los capítulos anteriores hemos abordado el diseño arquitectónico macroscópico. Sin embargo, en la fase final de implementación, la calidad del producto recae en la estructura microscópica del código fuente.

Según el plan de estudios del curso, un ingeniero de software no solo escribe código que funciona, sino código que es mantenible, escalable y legible. Esto se logra aplicando rigurosamente la filosofía de **Clean Code** y los **Principios SOLID**.

14.2. Filosofía Clean Code

El término *Clean Code* (Código Limpio), popularizado por Robert C. Martin (Uncle Bob), establece que el código debe ser escrito con la intención primordial de que sea **leído por otros seres humanos**, no solo compilado por máquinas.

Las heurísticas principales exigidas en la práctica son:

1. **Nombres con Significado Intencional:** Evitar variables como `int d;`. En su lugar, usar `int diasTranscurridos;`. Los nombres de clases deben ser sustantivos o frases nominales, y los nombres de métodos deben ser verbos o frases verbales.
2. **Funciones Pequeñas y de Propósito Único:** Una función debe hacer exactamente *una cosa*, hacerla bien y hacer *solo* esa cosa. Si una función ocupa más de 20 líneas o tiene múltiples niveles de anidamiento (`if/for` anidados), debe extraerse a submétodos.
3. **El Código se Auto-documenta:** El uso excesivo de comentarios es un *code smell* (olor a código estructurado deficientemente). En lugar de escribir un comentario explicando un bloque complejo, el ingeniero debe extraer ese bloque en una nueva función con un nombre tan descriptivo que el comentario se vuelva redundante.
4. **Manejo Excepcional de Errores:** Preferir el uso de *Excepciones* formales (`try/catch`) en lugar de retornar códigos de error matemáticos enteros (ej. `return -1;`).

14.3. Los Principios SOLID

SOLID es un acrónimo mnemotécnico que agrupa los cinco principios fundamentales de diseño orientado a objetos y programación ágil. Son la brújula para crear sistemas robustos frente al cambio.

- **S - Single Responsibility Principle (Responsabilidad Única):** Una clase debe tener **una y solo una razón para cambiar**. Si una clase maneja la lógica de facturación y, al mismo tiempo, imprime el recibo en un PDF, viola el principio. Deben separarse en `CalculadoraFactura` e `ImpresoraPDF`.
- **O - Open/Closed Principle (Abierto/Cerrado):** Las entidades de software (clases, módulos, funciones) deben estar **abiertas para su extensión, pero cerradas para su modificación**. *Ingeniería:* Cuando el cliente pida una nueva funcionalidad, el ingeniero debe poder añadirla agregando **nuevo** código (ej. creando una nueva subclase), sin tener que alterar el código matemático que ya funciona y está en producción.
- **L - Liskov Substitution Principle (Sustitución de Liskov):** Las clases derivadas deben poder ser sustituidas por sus clases base sin alterar la correctitud matemática del programa. Si una función espera recibir la superclase `Ave`, y le pasamos la subclase `Pinguino`, el programa no debe fallar repentinamente al llamar al método `volar()`.
- **I - Interface Segregation Principle (Segregación de Interfaces):** Es un error obligar a los clientes a depender de interfaces que no utilizan. *Ingeniería:* Es matemáticamente superior crear muchas interfaces pequeñas y especializadas (ej. `IImprimible`, `IExportable`) que una única interfaz "gorda" con métodos irrelevantes para quien la implementa.
- **D - Dependency Inversion Principle (Inversión de Dependencias):** Los módulos de alto nivel no deben depender de los módulos de bajo nivel; ambos deben depender de abstracciones (interfaces). Las abstracciones no deben depender de los detalles matemáticos, sino que los detalles deben depender de las abstracciones.

14.4. Otros Patrones Arquitectónicos Clave

Aunque el patrón de arquitectura en capas (Layered) y BCED son los pilares del curso, la profesora Fasolino también hace mención a otros patrones fundamentales que estructuran sistemas complejos:

- **Model-View-Controller (MVC):** Una variación clásica de la arquitectura en capas enfocada en la interacción de interfaces de usuario. Separa el **Model** (estado interno y lógica de negocio), la **View** (representación visual) y el **Controller** (manejo de eventos y actualizaciones).
- **Arquitectura Cliente-Servidor:** Separa la aplicación en componentes que solicitan servicios (Clientes) y componentes concurrentes que los proveen (Servidores), típicamente a través de una red.
- **Arquitectura SOA (*Service-Oriented Architecture*):** Basada en un componente central llamado **Broker** (intermediario). Los Servidores (proveedores de servicios) registran sus capacidades en el Broker, y los Clientes consultan al Broker para localizar e invocar dinámicamente esos servicios.

14.5. Automatización y Versionado: Git y GitHub

Para coordinar el trabajo en equipo y gestionar la escalabilidad de un proyecto, el curso estipula el uso obligatorio de herramientas de versionado distribuidas.

Definición 14.1 (Sistemas de Control de Versiones (VCS)). Un VCS registra metódicamente todos los cambios realizados en un conjunto de archivos a lo largo del tiempo para que se puedan recuperar versiones específicas más adelante. **Git** es el estándar de facto actual como VCS distribuido.

Conceptos Críticos de Versionado Distribuido:

- **Commit:** Un punto de guardado inmutable en el historial matemático del proyecto que contiene las diferencias (*diffs*) respecto al estado anterior.
- **Branch (Rama):** Una línea independiente de desarrollo. Permite a un ingeniero desarrollar una nueva *User Story* sin afectar a la rama de producción principal (*main* o *master*).
- **Merge:** La operación algorítmica de reintegrar una rama paralela dentro de la rama principal, resolviendo posibles conflictos de código concurrentes.
- **GitHub:** Es un servicio de alojamiento en la nube para repositorios Git. Permite la colaboración en remoto, las revisiones de código (*Pull Requests*) y la gestión del *Product Backlog* mediante *Issues*.

14.6. Herramientas de Evaluación Continua de la Calidad

Finalmente, la profesora evalúa en el proyecto la implementación de herramientas que cuantifiquen automáticamente los atributos de la calidad interna (vistos en el Capítulo 11 de la norma ISO 25010).

Idea Clave: El Concepto de Deuda Técnica (*Technical Debt*)

Es el coste implícito de un trabajo de programación adicional causado por elegir una solución "fácil" en el presente en lugar de usar un enfoque de ingeniería superior. Si la deuda no se refactoriza y paga, el software colapsa a largo plazo.

Para detectar la deuda técnica, se emplean tres herramientas analíticas fundamentales exigidas en el examen final:

1. **SonarQube / SonarCloud:** Servidor de inspección continua que realiza un análisis estático del código (*Caja Blanca*). Detecta vulnerabilidades de seguridad (*Bugs*), defectos lógicos, falta de cobertura de Testing (JUnit) y cuantifica la **Deuda Técnica** en horas de trabajo necesarias para arreglarla.
2. **SonarLint:** Un plugin o extensión directa para el IDE (como IntelliJ) que detecta y alerta al ingeniero de *Code Smells* y errores estructurales en tiempo real, mientras está escribiendo el código (similar a un corrector ortográfico, pero de diseño).
3. **ArchUnit:** Es una biblioteca de testing específica para la arquitectura en Java. *Aplicación de Ingeniería:* Mientras JUnit prueba que un algoritmo suma bien, **ArchUnit prueba que la arquitectura de diseño se cumpla matemáticamente**. Permite escribir un test automatizado que impida la compilación si detecta, por ejemplo, que una clase del paquete **boundary** (Vista) intenta acceder directamente a una clase del paquete **data** (Base de Datos), garantizando que nadie viole el patrón BCED accidentalmente.

Capítulo 15

Testing Estructural y Mantenimiento del Software

15.1. Testing Estructural (Caja Blanca)

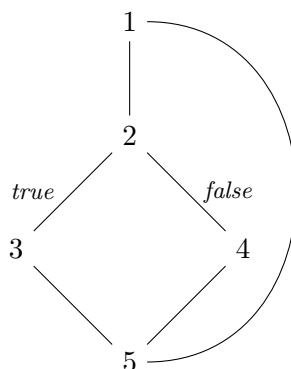
Mientras que el Testing Funcional (Caja Negra) deriva sus casos de prueba de las especificaciones de requisitos, el **Testing Estructural** utiliza la arquitectura y la topología interna del código fuente para diseñar los tests.

El objetivo matemático de la Caja Blanca no es verificar si el software cumple lo que el cliente pidió, sino responder a la pregunta empírica: «¿*Qué porcentaje exacto de mi código fuente ha sido ejecutado y validado durante las pruebas?*». Este enfoque completa al test funcional considerando elementos estáticos como instrucciones, decisiones, condiciones y caminos lógicos.

15.1.1. El Grafo de Flujo de Control (CFG)

Para aplicar rigor matemático al análisis del código, el programa se modela como un grafo dirigido $G = (N, E)$, conocido como **Control Flow Graph (CFG)**.

- **Nodos (N):** Representan bloques de sentencias secuenciales ininterrumpidas.
- **Aristas (E):** Representan transferencias lógicas de control (ramificaciones `if/else`, bucles `while`, llamadas a funciones).



15.1.2. Complejidad Ciclomática (Métrica de McCabe)

La profesora subraya la importancia de esta métrica. La **Complejidad Ciclomática** $V(G)$ es una medida de software que indica la complejidad lógica de un programa.

Idea Clave: Cálculo de la Complejidad Ciclomática

Dado un CFG, se calcula mediante la fórmula de la teoría de grafos:

$$V(G) = E - N + 2P$$

Donde E es el número de aristas, N el número de nodos, y P es el número de componentes conexos (habitualmente $P = 1$ para un solo programa).

Significado en Testing: $V(G)$ representa el **límite superior** del número de pruebas necesarias para garantizar la cobertura de todas las ramas lógicas independientes al menos una vez.

15.1.3. Criterios de Adecuación (Code Coverage)

Definición 15.1 (Test Adequacy - El Axioma Fundamental). Una suite de pruebas se considera "adecuada" respecto a un criterio cuando satisface una propiedad matemática observable de dicho criterio. **Axioma Crítico:** La adecuación *no indica que el programa sea correcto*, solo indica que la suite de pruebas cumple con la exigencia estructural seleccionada.

Existen varios niveles de exhaustividad:

1. **Cobertura de Sentencias (*Statement Coverage*):** Se diseña el conjunto mínimo de tests para asegurar que cada línea de código (nodo) se ejecute al menos una vez. Es el criterio más débil.
2. **Cobertura de Decisiones (*Branch Coverage*):** Exige que cada arista lógica del CFG (las salidas *true* y *false* de cada *if*) se recorra al menos una vez. *Matemáticamente: Cobertura de Ramas $\implies$ Cobertura de Sentencias.*
3. **Cobertura de Condiciones (*Condition Coverage*):** Exige que cada sub-expresión booleana individual asuma tanto el valor *true* como el *false* de forma independiente.
4. **Cobertura MC/DC (*Modified Condition/Decision Coverage*):** Vital para software de misión crítica (como normativas de aviación). Exige **demostrar que cada condición afecta de forma independiente al resultado final** de la decisión.
5. **Cobertura de Caminos (*Path Coverage*):** El criterio más exhaustivo. Exige probar todas las combinaciones de caminos a través del CFG. Debido a los bucles, este número tiende a infinito, haciéndolo inviable.

Ejemplo 15.2 (Caminos Inviabiles vs. Código Muerto). Un camino **Infeasible** (Inviabile) es aquel que topológicamente existe en el CFG pero que es matemáticamente imposible de ejecutar.

Ejemplo de la profesora: `if (x > 0) { B1 }; if (x < 0) { B2 }`; El camino secuencial que ejecuta B1 y luego B2 es *infeasible* porque ninguna variable x puede ser simultáneamente mayor y menor que cero.

Sin embargo, **esto no es código muerto**. B1 puede ejecutarse aisladamente con $x = 5$, y B2 con $x = -3$. **Código Muerto** es aquel que bajo *ninguna* circunstancia matemática se ejecutará (ej. `if (false) { B3 }`).

15.2. Automatización de Pruebas: JUnit y Mutation Testing

En la ingeniería moderna, los Casos de Prueba deben ser mecanizados para garantizar repetibilidad continua. **JUnit** es el framework estándar en el ecosistema Java. Utiliza *Test Drivers* (para instanciar y orquestar las clases) y *Mocks/Stubs* (objetos simulados) para aislar componentes durante el Unit Testing.

15.2.1. Límites de la Cobertura y Mutation Testing

Las herramientas de *Code Coverage* (como JaCoCo) integradas en el IDE son útiles porque mapean dinámicamente las áreas débiles. Sin embargo, la profesora subraya un peligro fundamental: **la medición de coverage no genera nuevos tests automáticamente**, ni garantiza la calidad de las aserciones (*¿está el test realmente comprobando algo o solo ejecutando líneas vacías?*).

Para medir la eficacia genuina de una test suite, la ingeniería recurre al **Mutation Testing**:

Idea Clave: Mutation Testing

Consiste en **introducir intencionadamente un error semántico** (un "mutante", como cambiar un `<` por un `>` en una instrucción) en el código fuente.

- Si la suite de pruebas falla al ejecutarse, se dice que el mutante ha sido **asesinado** (*killed*), validando la eficacia robusta del test.
- Si la suite de pruebas sigue pasando en verde a pesar del error inyectado, el mutante **sobrevive**. Esto evidencia una carencia estructural grave: los tests no detectan fallos lógicos reales.

15.3. Evolución y Mantenimiento del Software

El desarrollo inicial es solo la punta del iceberg. El software no es estático una vez entregado al cliente.

Definición 15.3 (Mantenimiento del Software). El proceso formal de modificación de un producto de software *después* de su entrega (*release*) para corregir fallos, mejorar atributos de calidad o adaptarlo a cambios en su entorno operativo.

Datos Empíricos Clave: Según las métricas de la industria, el mantenimiento absorbe típicamente el **70 % del presupuesto total** del ciclo de vida del software, e involucra al **75 % del personal** de ingeniería.

Observación 15.4 (La Metodología frente al Mantenimiento). - **Modelo Cascada (Waterfall):** Considera el mantenimiento como una fase *distinta y final*, aislada topológicamente del desarrollo inicial.

- **Metodologías Ágiles (Agile):** Borran la frontera temporal. El sistema se desarrolla de forma iterativa-incremental, por lo que desarrollo y mantenimiento (evolución) ocurren de forma concurrente y paralela.

15.3.1. Las Leyes de Evolución de Lehman

M. Lehman formuló axiomas empíricos sobre la dinámica del software en producción:

1. **Ley de Cambio Continuo:** Un programa utilizado en el mundo real debe cambiar obligatoriamente; de lo contrario, se volverá progresivamente menos útil en su entorno.
2. **Ley de Complejidad Creciente:** A medida que un programa evoluciona, su estructura interna se deteriora a menos que se invierta un esfuerzo proactivo en mantenerla (pagando la *Deuda Técnica*).

15.3.2. Tipología de la Manutención

La ingeniería clasifica las intervenciones post-entrega en cuatro categorías:

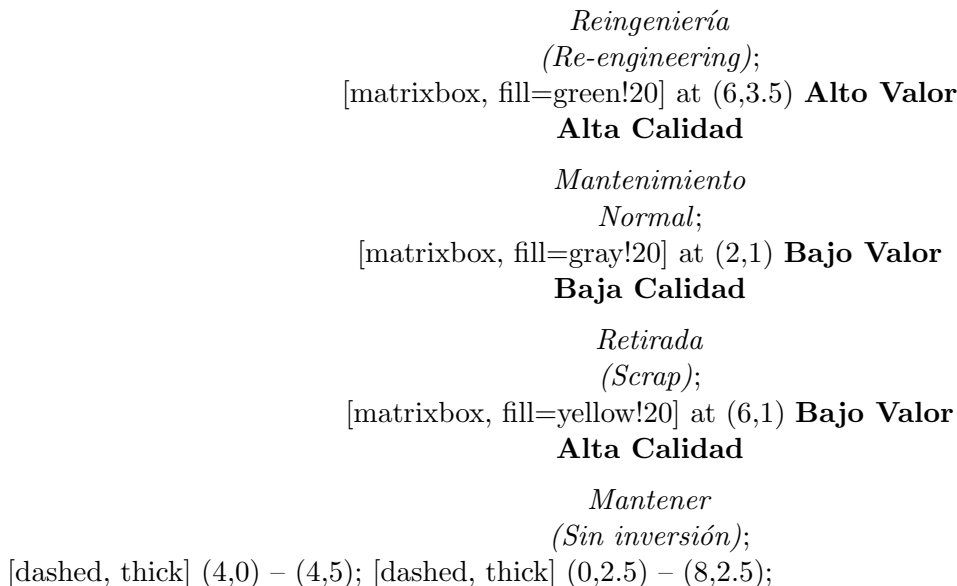
- **Mantenimiento Correctivo (Bug-fixing):** Intervenciones para reparar defectos y anomalías.

- **Mantenimiento Adaptativo:** Modificaciones dictadas por cambios en el entorno de alojamiento.
- **Mantenimiento Perfectivo (Evolución):** Alteraciones para añadir nuevas funcionalidades.
- **Mantenimiento Preventivo (*Refactoring*):** Se altera la estructura para reducir la Complejidad Ciclomática sin cambiar el comportamiento externo.

15.4. Rejuvenecimiento de Software y Sistemas Legacy

Definición 15.5 (Sistemas Legacy (Heredados)). Sistemas informáticos antiguos que siguen en producción y son críticos para el negocio de una organización, pero cuyo mantenimiento se ha vuelto económicamente insostenible o tecnológicamente obsoleto.

Para decidir qué hacer con un Sistema Legacy (mantenerlo, retirarlo o someterlo a rejuvenecimiento), las empresas cruzan métricas de *Valor de Negocio* frente a *Calidad Técnica* (incluyendo Comprensibilidad, Documentación y Rendimiento).



15.4.1. Técnicas para Rejuvenecer el Software (*Software Rejuvenation*)

Si un sistema se encuentra en el cuadrante de **Alto Valor / Baja Calidad**, se evita el enorme riesgo de programarlo desde cero aplicando el **Rejuvenecimiento de Software**, un ciclo transformacional de tres etapas:

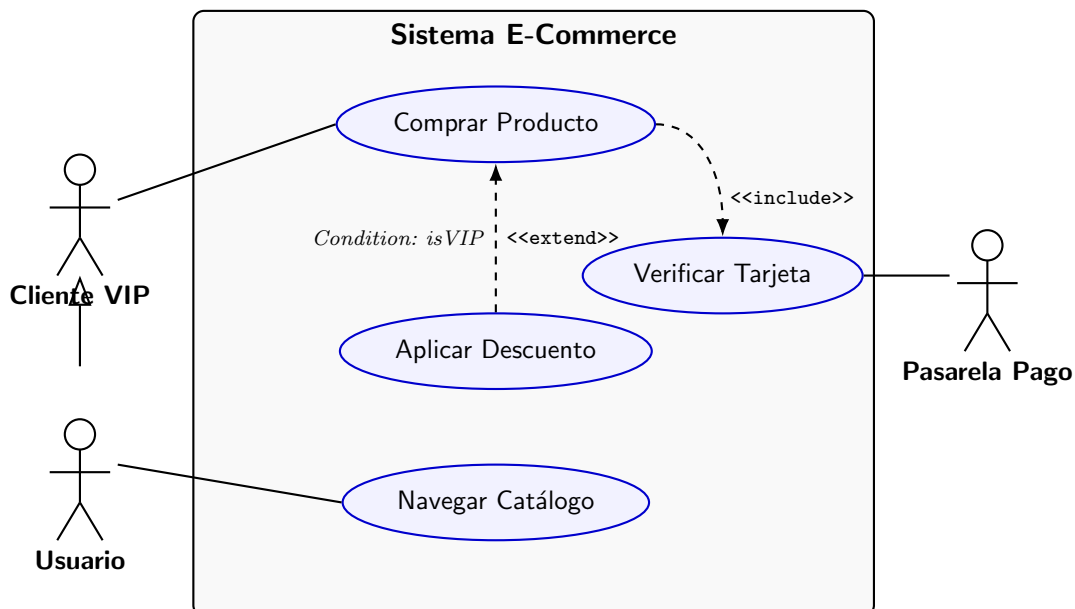
1. **Ingeniería Inversa (*Reverse Engineering*)**: Extraer modelos abstractos y topológicos a partir del código fuente antiguo incomprensible.
2. **Reestructuración (*Restructuring*)**: Transformar y limpiar el código sin cambiar su funcionalidad abstracta original (refactoring profundo).
3. **Ingeniería Directa o Reingeniería (*Forward Engineering*)**: Regenerar el nuevo código fuente (ej. migrar de lenguajes antiguos a modernos) a partir de los modelos limpios y reestructurados.

Capítulo 16

Compendio Práctico de Sintaxis UML

Este capítulo sirve como guía de referencia rápida (*cheat sheet*). Se presentan diagramas de alta densidad conceptual que concentran toda la sintaxis posible, acompañados de una leyenda analítica.

16.1. 1. Diagrama de Casos de Uso



Actor (Stickman): Entidad externa. A la izquierda suele ir el Actor Principal (inicia la acción) y a la derecha el Actor Secundario (provee servicios al sistema).

Frontera (Rectángulo gris): Delimita el alcance del sistema. Todo lo que está dentro es software a desarrollar; lo de fuera es el entorno.

Generalización de Actores (Flecha $\triangle$ hueca): Indica que un actor (Usuario VIP) hereda el rol de otro (Usuario general).

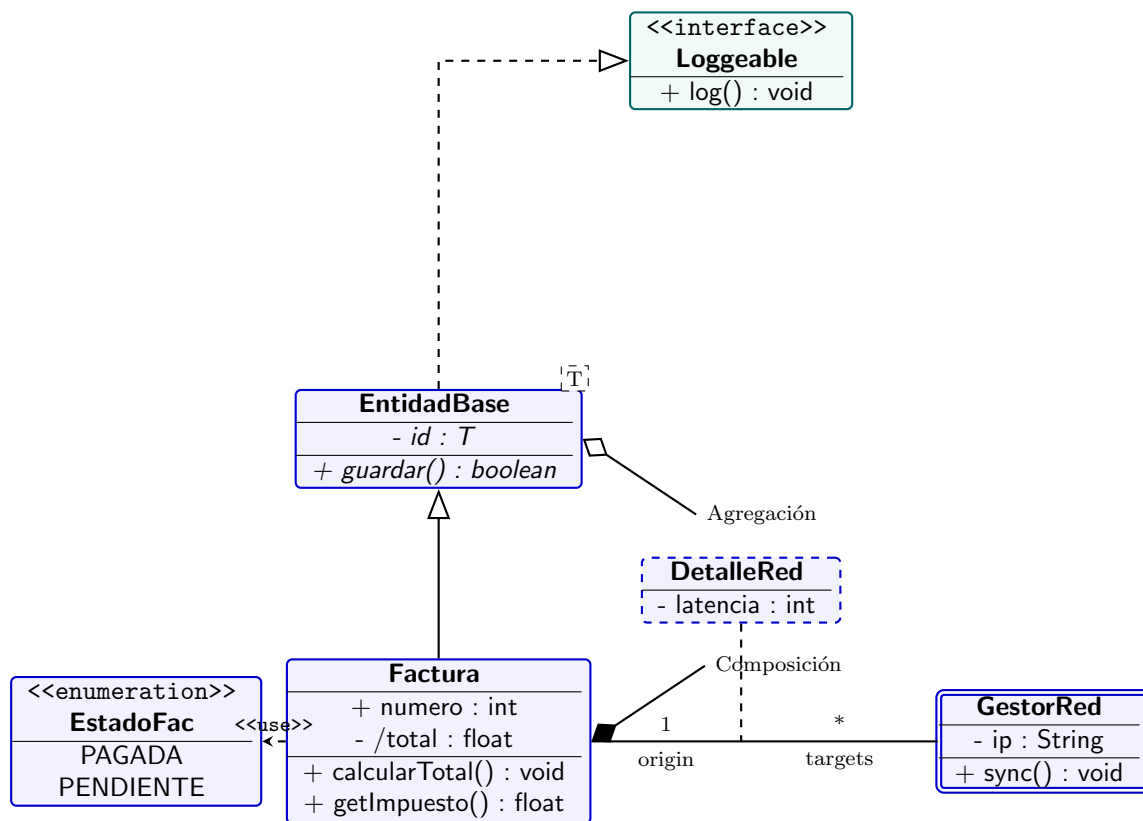
Caso de Uso (Elipse): Unidad de funcionalidad que aporta valor al actor.

Asociación (Línea continua): Indica que el actor y el caso de uso se comunican.

<<include>> (Flecha discontinua $\rightarrow$): Ejecución obligatoria. El caso base llama al incluido. La flecha apunta al incluido.

<<extend>> (Flecha discontinua $\rightarrow$): Ejecución opcional bajo una condición. La flecha apunta del extensor al caso base.

16.2. 2. Diagrama de Clases (Avanzado)



Nombre Cursiva: Indica que la clase o el método es **abstracto**.

<<interface>>: Un contrato de comportamiento sin estado. La flecha punteada con triángulo hueco indica **Realización** (implements).

Cuadro discontinuo superior derecho: Indica una clase **Paramétrica** (Template/Generics, ej. en Java `EntidadBase<T>`).

Doble borde lateral: Indica una **Clase Activa** (posee su propio hilo de ejecución/Thread).

Visibilidad: + (Public), - (Private), # (Protected), ~ (Package).

Atributo Derivado (/total): Su valor no se almacena, se calcula matemáticamente a partir de otros.

Subrayado (getImpuesto()): Indica un atributo o método estático (*class level*).

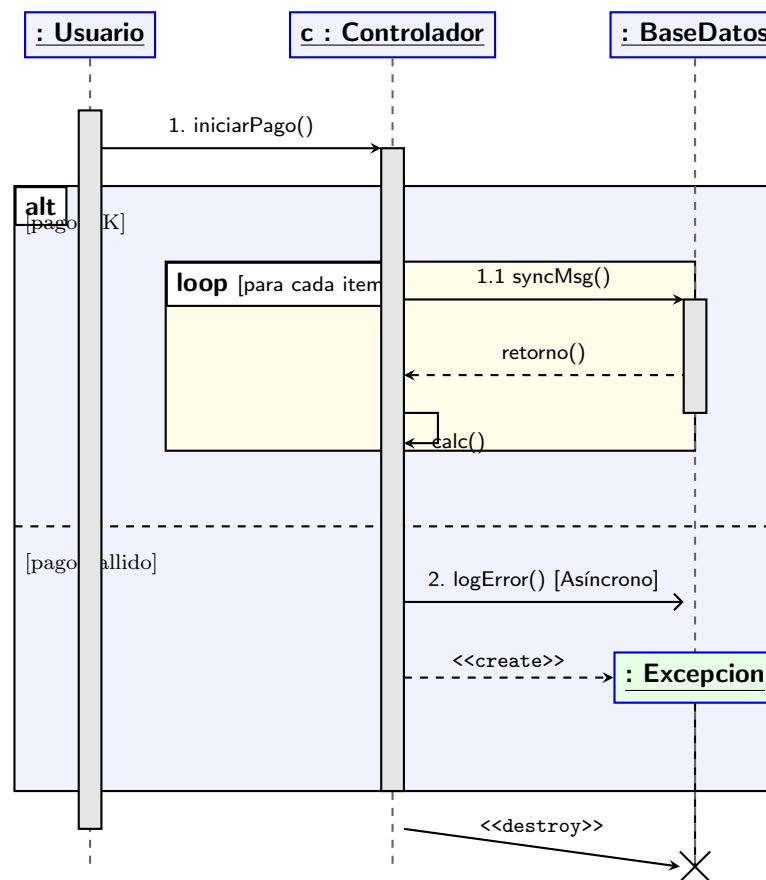
Clase Asociativa (Línea discontinua a una asociación): Atributos que pertenecen a la relación $N : M$, no a las clases individuales.

Dependencia (<<use>>, flecha → discontinua): Relación efímera (ej. uso de una enumeración o paso de parámetro).

Composición (Rombo relleno ◆): Relación Todo-Parte de vida estricta. Si muere el Todo, muere la Parte.

Agregación (Rombo hueco ◇): Relación Todo-Parte flexible. Las partes sobreviven a la destrucción del Todo.

16.3. 3. Diagrama de Secuencia



Eje X e Y: El tiempo avanza hacia abajo. Las columnas representan objetos anónimos o instanciados.

Caja de Activación (Rectángulo gris): Muestra el *Foco de Control*, el tiempo que un objeto tiene la CPU.

Flecha → (Punta rellena): Mensaje **Síncrono**. El emisor se bloquea esperando respuesta.

Flecha → (Punta abierta): Mensaje **Asíncrono**. El emisor lanza el evento y sigue su ejecución.

Flecha ←-- (Discontinua): **Retorno** de datos tras finalizar una llamada síncrona.

Flecha <<create>>: Instanciación dinámica. Apunta a la *caja* del objeto, indicando que nace en ese instante *t*.

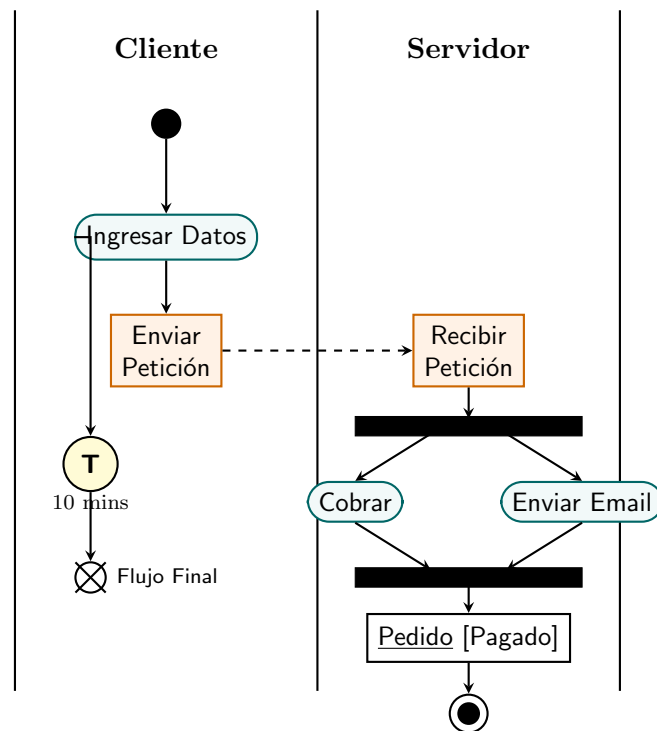
Cruz × (<<destroy>>): Destrucción del objeto. Su memoria es liberada y su línea de vida termina.

Auto-mensaje: Flecha que sale y vuelve a la misma línea de vida (invocación de un método privado local).

Fragmento alt: Simula un if-else. Se divide con una línea discontinua; solo un bloque se ejecuta.

Fragmento loop: Ejecuta repetidamente los mensajes internos mientras se cumpla la guarda.

16.4. 4. Diagrama de Actividades



Swimlanes (Carriles): Particionan el diagrama asignando formalmente la **responsabilidad** de ejecutar las acciones a un actor, clase o departamento específico.

Punto Inicial (●) y Final (⊙): Inicio topológico del proceso y final global (destruye todos los *tokens*).

Final de Flujo (⊗): Destruye el *token* de ese camino concreto sin abortar todo el proceso paralelo.

Barra Negra (Fork / Join): Modela concurrencia estricta. **Fork** (1 entrada → N salidas) clona el token en hilos paralelos. **Join** (N entradas → 1 salida) sincroniza los hilos esperando a que todos lleguen.

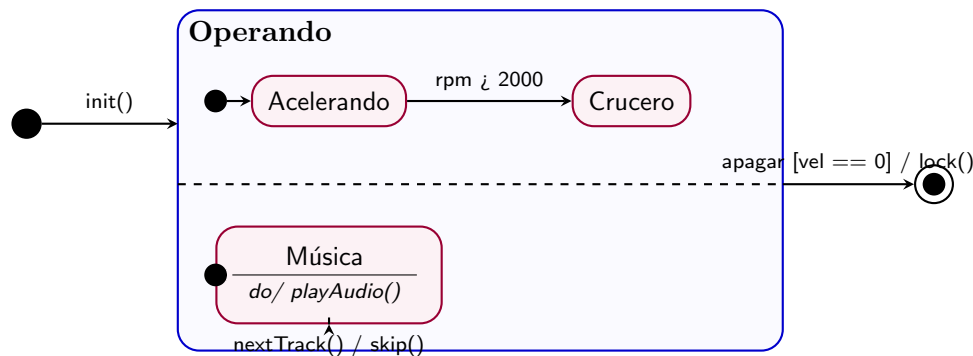
Señal de Envío (Polígono >): Emite un evento asíncrono hacia el exterior y continúa inmediatamente.

Recepción de Señal (Polígono <): Pausa la ejecución del token actual hasta que el evento externo sea capturado.

Evento Temporal (T): El flujo se detiene aquí durante un lapso de tiempo o hasta una fecha exacta.

Nodo de Objeto (Rectángulo con [Estado]): Representa cómo los datos fluyen y mutan de estado a lo largo del proceso.

16.5. 5. Diagrama de Máquina de Estados



Estado (Rectángulo de bordes muy redondeados): Condición en la que el objeto satisface un predicado. Puede incluir acciones: *entry/* (al entrar), *do/* (acción iterativa o continua), *exit/* (al salir).

Transición y su Sintaxis: Flecha que indica el cambio de estado. La sintaxis matemática estricta es:

Evento [Condición de Guarda] / Acción

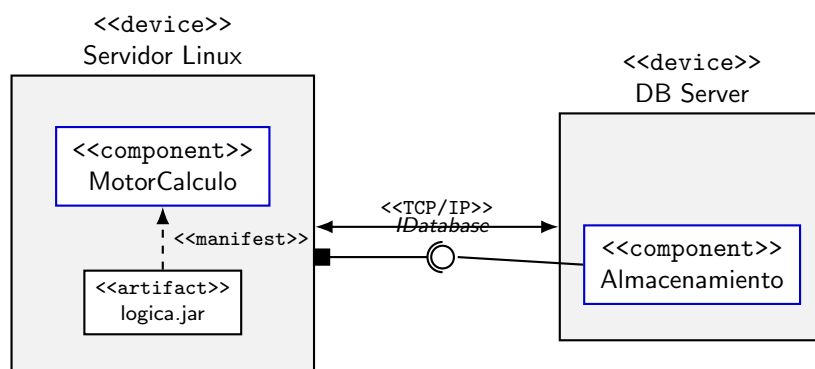
Si ocurre el *Evento*, pero la *Condición* (booleana) es falsa, la transición no se dispara. Si es verdadera, ocurre la transición y se ejecuta automáticamente la *Acción*.

Estado Compuesto: Un estado que contiene toda una sub-máquina de estados en su interior. Ayuda a reducir el caos visual de transiciones (todas las transiciones que salen del borde exterior del estado compuesto aplican a todos sus sub-estados).

Regiones Ortogonales (Línea discontinua): Indica **conurrencia**. Al entrar en el estado compuesto, se ejecutan de manera asíncrona y simultánea las máquinas de estados de todas las regiones internas separadas por las líneas discontinuas.

16.6. 6. Diagrama de Componentes y Despliegue

A menudo, la macroarquitectura se modela de forma híbrida: mostrando qué piezas lógicas (Componentes) residen en qué infraestructura de red/hardware (Despliegue).



Nodo <<device>> (Cubo 3D): Hardware físico con capacidad de procesamiento (CPU, RAM).

Nodo <<execution environment>>: Entorno de software (como una JVM o Docker) que corre dentro de un dispositivo.

Enlace de Comunicación: Línea que modela la red física o protocolo (ej. TCP/IP) entre nodos.

Artefacto (<<artifact>>): Archivo físico (ej. .jar, .dll, script).

<<manifest>> (**Flecha discontinua**): Indica que un archivo físico concreto (Artefacto) es la implementación compilada en el disco de un Componente lógico de UML.

Componente (Caja con icono/texto): Módulo reemplazable que encapsula comportamiento.

Puerto (Cuadrado en el borde): Punto de interacción formal que atraviesa la encapsulación del componente.

Lollipop (Círculo o): Provided Interface. La interfaz que el componente expone y ofrece al resto del mundo.

Socket (Semicírculo $\supset$): Required Interface. La interfaz que el componente necesita obligatoriamente de otro componente para poder funcionar.